



ENTRY &



PYTHON



Python bilan sayohat qilamiz!



RoboticsWare

Kirish

Ushbu kitob, Entry blokli kodlashni to'liq o'rganib olib keyin Python matnli kodlashga o'tishni istaydigan o'quvchilar va ularga bunda yordam berishni maqsad qilgan o'qituvchilar uchun yozilgan. Entry-Python o'quvchilarga blokli kodlashdan amaliy matnli kodlashga qiyinchiliksiz o'tishda yordam beradigan ko'prik bo'lib xizmat qiladi.

Nashriyot: **Roboticsware**

Muallif: JeongJun Lee

O'zbek tiliga tarjima: Komron (knmv2311@gmail.com)



Malumot materiallari: **Entry Python kitobning 1-versiyasi**

Mualliflik huquqi: <https://creativecommons.org/licenses/by-nc-sa/4.0/>



Mundarija

Kirish	2
1. Entry vs Entry-Python	4
2. Entry-Python bilan ishlashni boshlash	6
3. Entry-Python yordamida Python sintaksisining asoslarini o'rganamiz.....	8
3.1 "Hello World" namuna kodini tushunish Entry-Python.....	9
3.2 Kiritish/Chiqarish (Input/Output)	16
3.3 O'zgaruvchi (Variable)	19
3.4 Shart (Condition)	23
3.5 Takrorlash (Loop)	29
3.6 Ro'yxat (List)	32
3.7 Tasodifiy son (Random)	38
3.8 Funksiya (Function)	41
4. Entry-Python yordamida dasturlashning asosiy tushunchalarini o'rganamiz	45
4.1 Ketma-ketlik/Parallel bajarish (Serial/Parallel)	46
4.2 Jarayonga yo'naltirilgan (Procedural)	49
4.3 Hodisaga asoslangan (Event-Driven)	50
4.4 Obyektga yo'naltirilgan (Object-Oriented).....	52
* Ilova.....	55
Entry-Pythonning tezkor tugmalari.....	55

1. Entry vs Entry-Python

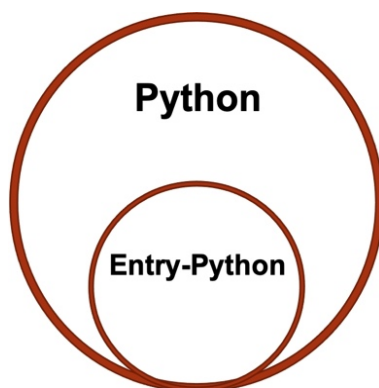
Entry - Naver bilan bog'langan [Connect Foundation](#) nodavlat notijorat tashkiloti tomonidan ishlab chiqilgan va homiylik qilingan ochiq manbali ta'lim kodlash tilidir ([EPL: Educational Programming Language](#)). Xususan, Koreyada bu yuqori darajali blokli kodlash tili (a high-level block-based visual programming language, yuqori darajadagi bloklarga asoslangan vizual dasturlash tili) dasturlashni o'rgatishda eng keng qo'llaniladi.

Dunyoda allaqachon eng mashhur bo'lgan [MIT Scratch](#) bilan solishtirganda, shu bilan eng katta ustunligi u bolalarni blokli kodlash orqali dasturlash dunyosiga kiritib natijada ularni matnli dasturlash(text-based programming)gacha kiritishni oldin mo'ljallanganidir. Entry matnli dasturlash uchun Python tilini tanlagan. Python tili matnli dasturlash tillari orasida o'rganish eng oson til bo'lib, ayniqsa, yaqinda e'tibor markazida bo'lgan ma'lumotlarni tahlil qilish va sun'iy intellekt sohalarida foydalanishga qulay tildir, shuning uchun u umumiy tarzda birinchi matnli dasturlash tilini o'rganishda eng ko'p tavsiya qilinadigan tildir.

Piton(Python): Python 1991-yilda gollandiyalik dasturlash muhandisi Guido van Rossum tomonidan chiqarilgan yuqori darajadagi dasturlash tili. Bu, [interpretator](#) dan ishlatadigan [obyektga yo'naltirilgan](#) til va platformadan mustaqil, [dinamik tipli](#) interaktiv til. Pythonning kuchli kutubxonalari va boy ekotizimlari yordamida siz ma'lumotlarni to'plashingiz, tahlil qilishingiz va vizualizatsiya qilishingiz mumkin. Pythonning ma'lumotlarni tahlil qilish sohasida keng qo'llanilishining sabablaridan biri uning boshqa dasturlash tillariga nisbatan oson va soddaligidir.

Manba: Wikipedia

Biroq, Entryda o'rnatilgan "Entry-Python" faqat Entry uchun mo'ljallangan Python tili deb aytish mumkin. Bu asl Python tilining asosiy grammatikasini o'rganish uchun o'quv maqsadlarida asl Pythonning kichik versiyasi sifatida ko'rib chiqilishi mumkin va shuning uchun u to'liq Pythonning o'zi emasligi sababli, kodlashda qo'llab-quvvatlanmaydigan asl Python sintaksisi mavjudligini doimo esda tuting.



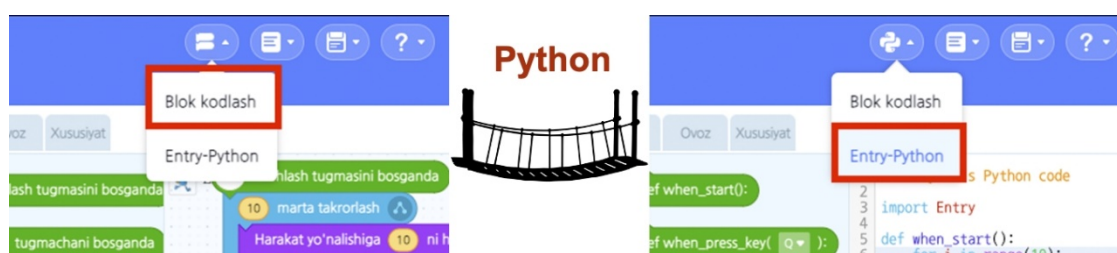
Bizning maqsadimiz “Entry-Python” orqali iloji boricha tezroq Pythonning asosiy sintaksisini o‘zlashtirib, imkon qadar tezroq [asl Python](#) ga o‘tish va blokli kodlashni butunlay tark etib, endilikda faqatgina matnli kodlash (dasturlash) bilan shug‘ullanishdir. Shu sababli, Entry-Python ichida juda katta umid va maqsaddan oshib ketadigan darajada foydalanish ruhiy salomatlik uchun yaxshi bo‘lmasligi mumkin. Buning sababi shundaki, afsuski, Entry-Python o‘zi kutilganidan ko‘ra mukammal emas (masalan, kodda sintaksis xatolari bo‘lmasa ham, xatolik borligini ko‘rsatib, dastur ishlamay qolishi kabi holatlar; buni mashq bajarib ko‘rsangiz, tushunasiz). Shuning uchun biroz murakkab kodlashni boshlaganingizda bunday muammolarga duch kelish ehtimoli oshadi.

Xulosa qilib aytganda, Entry-Python blokli kodlashdan matnli kodlashga o‘tishni endigina boshlaganlar uchun mo‘ljallangan bo‘lib, matnli kodlash dunyosiga kirib, dastlabki qiyinchiliklarni yengillashtirish uchun yaratilgan. O‘zimizga tanish bo‘lgan blokli kodlash muhiti ichida klaviatura orqali matnli kodlashni sinab ko‘rib, u blokli kodlashdan uncha farq qilmasligini tushunish va shuning evaziga matnli kodlash unchalik murakkab emas degan noto‘g‘ri tasavvurni yo‘qotish Entry-Pythonning asosiy maqsadiga erishish bo‘ladi.

2. Entry-Python bilan ishlashni boshlash

Entry (blokli kodlash) <-> Python(matnli kodlash)ga aylantirish

Agar siz allaqachon Entry foydalanuvchisi bo'lsangiz, Entry haqida oldindan ma'lumotga ega bo'lganingiz yoki Entry qanday ishlashini o'rganayotganda tasodifan bu menyuga duch kelgan bo'lishingiz mumkin. Ikki xil kod o'rtasida almashish uchun Entryning yuqori menyusidagi kodni o'zgartirish belgisini bosish kifoya.



Shu tarzda Entry nafaqat blokli kodlashni matnli kodlash tili (Python) sifatida qanday o'zgarishini kod konvertatsiyasi orqali ko'rish imkonini beradi, balki Entry-Python kodlash muharriri oynasi orqali to'g'ridan-to'g'ri matnli kod yozish va uni ishga tushirish funksiyasini ham taqdim etadi. Bunday Entry-Python o'quvchilar yoki o'qituvchilar uchun Entryda "blokli kodlashdan boshlab, matnli kodlashni o'zlashtirish" maqsadiga erishishda o'rtada bog'lovchi ko'priklarni bajaradi, deb aytish mumkin.

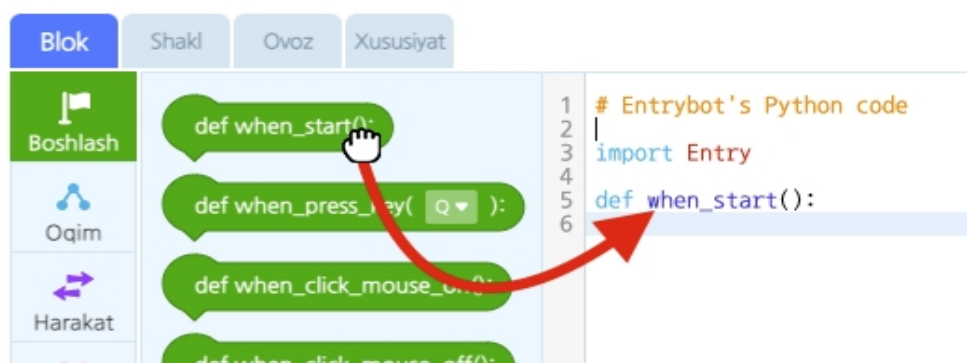
Blokli kodlashdan matnli kodlashga o'tish tajribasi

Odatda, blok dasturlash dunyosidan matnli dasturlash dunyosiga o'tishda qiynaladigan sababi oldindan bloklarni endi bloklarning bo'limidan muharrir(kodni yig'ish) maydoniga sudrab olib tashlashning intuitiv va qulay usulidan foydalanib dasturlash imkoniyati yo'q bo'lib qoladi. O'riniga endi siz matnning har bir belgisini qo'lda kiritib, so'zma-so'z "uzun insho yozishingiz" kerak. Ushbu jarayonda biz blokli dasturlashga xos bo'lmagan xatolarga duch kelamiz va ba'zida bu kichik ko'rinadigan xatolar (yoki errorlar) dasturning umuman ishga tushmasligiga olib keladi, bu esa biroz chalkashliklarni keltirib chiqaradi. Blokli dasturlashda, agar kod noto'g'ri yozilgan bo'lsa ham, dastur shunchaki noto'g'ri ishlaydi, lekin to'liq ishlashni to'xtatmaydi.

Matnli dasturlashdagi xatolar matn terish xatolaridan kelib chiqishi mumkin, bu juda tushunarli, lekin ba'zida muammo faqat katta va kichik harflar orasidagi farq yoki nuqta mavjudligi/yo'qligi bilan bog'liq. Dasturni ishga tushirishga urinayotganda, hatto bunday mayda narsa ham xato deb hisoblanishi mumkin va dastur ishga tushirishni rad etadi. Matnli dasturlash kodni juda ehtiyotkorlik bilan va aniq yozishni, shuningdek tanlangan dasturlash tili uchun belgilangan sintaktik qoidalarga rioya qilishni talab qiladi. Shu sababli, matnli dasturlashdagi birinchi qiyinchiliklar har bir kishi uchun tabiiydir va, albatta, ushbu dasturlash uslubiga ko'nikish uchun vaqt kerak bo'ladi.

Entry-Python da olib tashlash usuli (Drag&Drop) yordamida dasturlash

Entry-Pythonda matnli dasturlashda matn terish xatolaridan kelib chiqadigan xatolarni oldini olish uchun blokli kodlashdan olingan fikrdan foydalanish mumkin. Rasmda ko'rsatilgandek, blokli kodlashda bo'lgani kabi, sudrab olib tashlash (Drag&Drop) orqali dasturlashga urinib ko'rishingiz mumkin.



Bu usul qulay, chunki u oddiy matn terish xatolaridan kelib chiqadigan xatolarni kamaytiradi, shuningdek, klaviatura terish vaqtini qisqartiradi. Biroq, yuqorida aytib o'tilganidek, uni ishlatish uchun siz tanlangan dasturlash tilining sintaktik qoidalarini oldindan yaxshi bilishingiz kerak. Bu shuni anglatadiki, ushbu yondashuv faqat dasturlash tili sintaksisining asoslari bilan tanish bo'lgan taqdirdagina qo'llanilishi mumkin. Shuning uchun bizning asosiy vazifamiz Python sintaksisining asosiy qoidalarini o'rganishdan boshlashdir. Keyingi bo'limlarda biz ularni bosqichma-bosqich o'rganamiz.

3. Entry-Python yordamida Python sintaksisining asoslarini o'rganamiz

Matnli dasturlashning grammatik tizimini o'rganish chet elliklar bilan muloqot qilish uchun chet tilini o'rganish jarayoniga o'xshaydi. Biroq, suhbatdosh chet ellik emas, balki kompyuter. Xuddi chet tilini birinchi marta o'rganganimizda, biz o'sha tilni ifodalovchi harflarni bilishdan boshlaymiz (baxtimizga, kompyuterlar bilan suhbatlar faqat ingliz tiliga asoslangan lotin alifbosida yoziladi, buni biz allaqachon bilamiz), va faqatgina harflar bo'g'inlarni, bo'g'inlar so'zlarni tashkil etganda, o'sha so'zlar gaplar tuzuvchi grammatika tizimiga (masalan, ot-sifat-fel tartibi va boshqalar) muvofiq tartibga solinsagina, biz bir-birimiz bilan suhbatlasha olamiz.

Bizda chet tilini o'rganishda tajriba bor (hech bo'lmaganda ingliz tili va h.k.) va bu biz uchun afzallik bo'lishi mumkin, lekin shu bilan birga, bizda til o'rganish qiyin degan notog'ri fikr ham bo'lishi mumkin. Biroq, kompyuter dasturlash tillari odamlar tomonidan ishlatiladigan tillarning murakkabligiga qaraganda ancha oson va ular orasida Python eng oson kompyuter dasturlash tillaridan biri hisoblanadi. Shuning uchun siz boshidan qiynalmaysiz degan umiddaman.

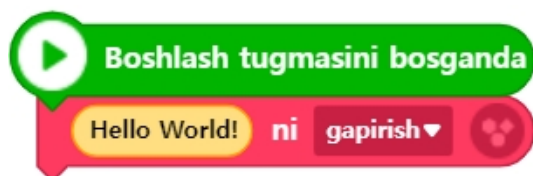
3.1 “Hello World” namuna kodini tushunish | Entry-Python

Agar boshqa turdagi kompyuter dasturlash tillari kitoblarini o‘qib ko‘rgan bo‘lsangiz, ekranda “Hello World!” degan oddiy jumlaning chiqaruvchi kod misoli bilan boshlanishini ko‘rgan bo‘lishingiz mumkin. Biz ham ushbu odatiy misol koddan boshlab, grammatikani o‘rganishni boshlaymiz.

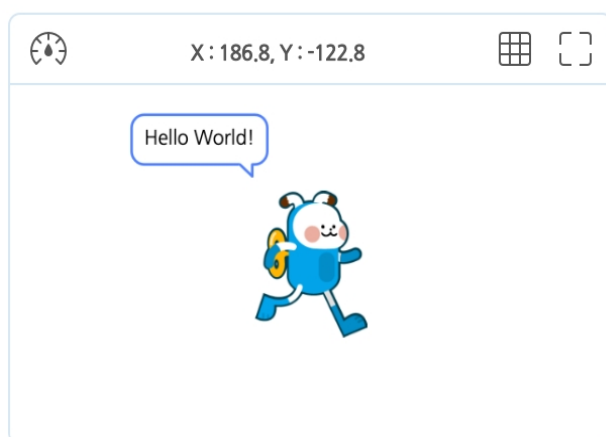
Entry-Python

```
1. # Entrybot's Python code
2.
3. import Entry
4.
5. def when_start():
6.     Entry.print("Hello World!")
```

Blokli kodlash

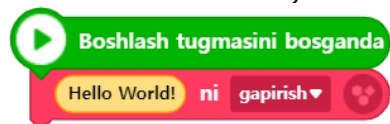


Ijro narijasi



Kelgusida kod tahlili yuqoridagi kabi 3 ta yorliq shaklida tuziladi: birinchi yorliq kodning bajarilish natijasini ko‘rsatadi, ikkinchi va uchinchi yorliqlarda esa bir xil bajarilish natijasini olish uchun Entry-Python va blokli dasturlash shaklidagi kodlarni taqqoslash imkoniyati yaratiladi.

Ushbu misolda bu juda oddiy dastur bo'lib, uni blok kodlashda faqat ikkita blok



bilan kodlash mumkin. Biroq, Entry-Python bilan kodlaganda, u 6 qatordan iborat uzun kodga aylanadi (kodni o'qishni yaxshilash uchun o'rtaga kiritilgan bo'sh qatorlar ham hisobga olingan). Men hozir bu misolni o'zingiz yozishingizni tavsiya etmayman. Keyinchalik kodlash uchun ko'plab imkoniyatlarga ega bo'lasiz va birinchi navbatda matnli dasturlash misoli sifatida eng oson kod deb ataladigan ushbu 6 qatorning ma'nosini aniq tushunish muhimroqdir. Shuning uchun men ushbu tarkibni hozircha blok kodlash bilan kodlashni tavsiya qilaman va keyin uni Entry-Python kodiga aylantirish uchun menyuda avtomatik kod o'zgartirish menyusini tanlang.

1-qatordagi “# Entrybot's Python code” nimani anglatadi? **Matnli dasturlashda siz har bir belgi, hatto ba'zan bo'sh joy ham ma'no va maqsadga ega ekanligini bilasiz.** Demak, 1-qatorda birinchi bo'lib paydo bo'lgan # belgisi ham ma'no va maqsadga ega ekanligini xulosa qilishimiz mumkin va uni joriy # belgisidan keyin yozilgan tarkibdan xulosa qilishimiz mumkin. Bu “Ushbu belgidan boshlab va undan keyin (lekin bir qator ichida) yozilgan hamma narsani izoh (comment, komentariya) deb hisoblang” degan ma'noni anglatadi.

Matnli kodlashda **izoh (comment, komentariya)** degan tushunchaning ma'nosini tushunib olaylik. Kompyuter biz yozgan kodni yakuniy bajarish uchun bosqichma-bosqich o'qib chiqadi. Ammo kodda izoh qismiga duch kelganda, uni bajarish jarayoniga hech qanday aloqasi yo'qligi sababli uni o'qimay, e'tibordan chetda qoldirishi kerakligi haqida bizdan “xabar” oladi. Shu bois, kompyuter ushbu qismni o'qishdan voz kechib, uni chetlab o'tadi.

Xo'sh, **izoh nima uchun kerak? Uning ikkita asosiy maqsadi bor:** birinchisi, izohning nomi o'zi aytib turganidek, yozilgan kodni tushuntirishdir. Ya'ni, qo'shimcha ma'lumot berish orqali keyinchalik muallifning o'zi yoki bizning hamkasblarimiz ushbu kodni ko'rib, uni tez va to'g'ri tushunishiga yordam berish uchun ishlatiladi. Ikkinchisi, kodda mantiqiy xatolarni (sintaksis xatolar emas, balki bajarilganda kutilganidek ishlamaydigan xatolarni) tuzatish jarayoni (Debugging – xatolarni topish va tuzatish jarayoni)da ishlatiladi. Bu jarayonda, kodning bir qismini vaqtincha izoh sifatida belgilab, uni bajarishdan chetda qoldirish (o'tib ketish) orqali xatoni tezroq aniqlash uchun ishlatiladi.

Endi 2-qator tahliliga o'tamiz. Bu qator oddiygina bo'sh qoldirilgan. Bunday qatorlar 2- va 4-qator bo'lib, umumiy ikki qatorni tashkil etadi. Nega bo'sh qator kiritilgan? Sababi juda oddiy. Kompyuter uchun kodni tahlil qilishda hech qanday ahamiyatga ega emas, lekin biz, insonlar, kodni o'qiganimizda uni yaxshiroq va tushunarliroq qilish uchun kiritiladi. Garchi hozir matnli kodlashni endigina boshlayotganimiz bois buni yaxshi anglamasakda, dasturlashning bir oz kattaroq loyihalariga o'tganimizda, odatda yakkama-yakka emas, balki boshqa dasturchilar bilan hamkorlikda ishlashimiz kerak bo'ladi. Shu bois, boshqa odamlar sizning kodingizni oson, tez va qulay o'qiy olishi uchun doim e'tiborli bo'lish odatini rivojlantirish muhimdir. Bu dasturchilar uchun zarur

bo'lgan asosiy fazilatlardan biridir. Shuning uchun kod yozishda tegishli bo'sh qatorlardan foydalanish va tegishli joyda izoh kiritish juda muhim.

¹²₃₄ Endi **3-qatorga** o'tamiz. Unda **import Entry** degan ikki so'z bor. Bu jumlada **import** kalit so'zining ma'nosi – biz kod yozayotgan faylga tashqi manbadan kutubxona(Library, Pythonda paket(Package), undan kichikroq birlik – modul(Module) deb ataladi)ni olib kelib, undan foydalanishni bildiradi. **import** dan keyin keladigan **Entry** so'zi esa bizga kerak bo'lgan kutubxonaning nomi. Ushbu kodning umumiy ma'nosi shundan iboratki, **Entry** (kutubxonaning nomi kutubxona muallifi tomonidan beriladi) deb nomlangan kutubxonani tashqi manbadan chaqirib olib foydalanishimiz kerak. Endi kutubxona nima ekanligini bilib olish vaqti keldi.

Kutubxona nima?

Kutubxona (Library): Dasturiy ta'minotni ishlab chiqish da kompyuter dasturi tomonidan foydalaniladigan o'zgarmas resurslar to'plami. Bunga konfiguratsiya ma'lumotlari, hujjatlar, yordamchi materiallar, xabarlar shablonlari, oldindan yozilgan kodlar , subrutinlar (funksiyalar), klass lar, qiymat lar va ma'lumot turlari , spetsifikatsiyasi kiradi.

Manba: Vikipediya

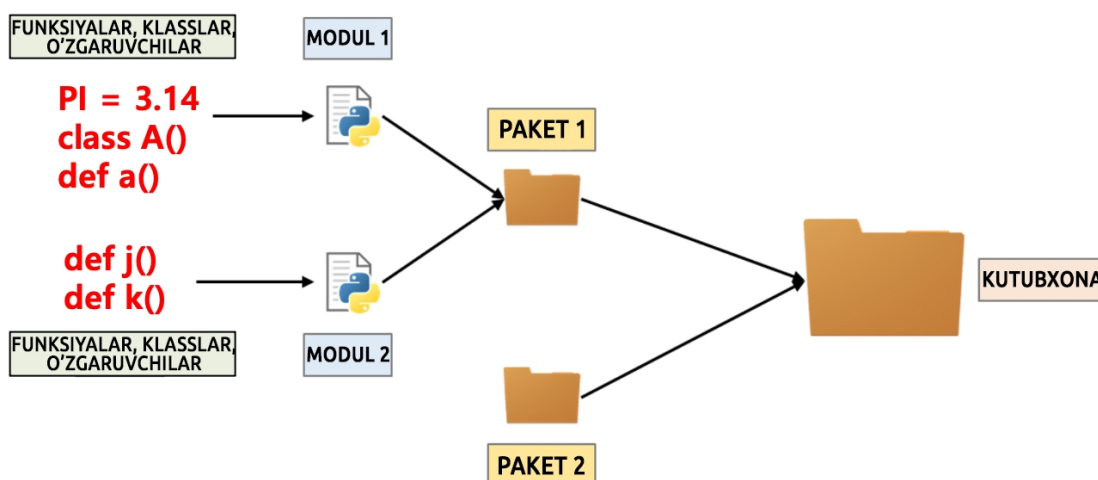
Yuqoridagi kutubxona ta'rifini o'qib tushundingizmi? **Kutubxonani oddiy qilib tushuntiradigan bo'lsak, bu dasturchilar tomonidan dasturlarni tez va qulay ishlab chiqish maqsadida oldindan yozib qo'yilgan kodlar to'plami (asosan funksiyalar, klasslar, o'zgaruvchilar va boshqalar)** deb aytish mumkin. Siz allaqachon Entryda funksiyalardan foydalanib ko'rgansiz va funksiyaning maqsadini yaxshi tushunasizmi? Funksiyaning asosiy maqsadi nima edi? Bir nechta joyda takroran ishlatiladigan kod qismlarini funksiya sifatida ajratib yozib qo'ysak, keyinchalik ushbu funksiya blokini bir marta chaqirish orqali xohlagan vazifamizni osongina bajarishimiz mumkin bo'ladi. Natijada kodlash jarayoni sezilarli darajada soddalashadi, kod samaradorligi oshadi va funksiyalar tufayli soddalashtirilgan kod butun dasturning tushunilishini sezilarli darajada yaxshilaydi.

Bundan tashqari, **funksiyaning yanada foydali va asosiy maqsadi — yaxshi yozilgan funksiyani dasturdagi turli joylarda (bitta obyekt yoki ko'p sonli obyektlar ichida) bir necha marta ishlatishdir (buni kodni qayta ishlatish deyishadi).** Bu dastur bir marta yozilib tugatilmay, balki doimiy ravishda yangilanish va rivojlanish jarayonini boshdan kechiradigan tirik mavjudot kabi bo'lgani sababli, keyingi o'zgarishlarga osonlik bilan moslashishga imkon beradi. Chunki biror funksiyani alohida bir vazifa uchun ajratib, yozib qo'yganingizdan keyin, o'sha funksiyaning mazmunini yaxshilash yoki o'zgartirish kerak bo'lsa, ushbu funksiyadan foydalanilgan barcha kodlarda o'zgarishlar bir vaqtning o'zida avtomatik ravishda tatbiq etiladi.

Agar hozirda funksiyaning maqsadi va uning mazmunini to'liq tushunmagan bo'lsangiz, ehtimol siz Entry blokli kodlashda funksiyalardan foydalanib ko'rmagan yoki

funksiyaning maqsadini yaxshi tushunmasdan foydalangan bo'lishingiz mumkin. Bu yerda funksiya haqida qayta tushuntirish bu kitobning doirasiga kirmaydi, shuning uchun kimga kerak bo'lsa [ma'lumot havola](#) ni qoldiraman, ularni o'zlari mustaqil ravishda o'rganishlari mumkin.

Agar bir yaxshi yozilgan funksiyaning qanchalik foydali ekanligini tajribada ko'rgan bo'lsangiz, endi shunday savol tug'ilishi mumkin: "Bu funksiyaning faqat o'zim ishlata olishim juda achinarli emasmi? Buni barcha dasturchilar, kichikroq miqyosda mening hamkasblarim yoki butun dunyo dasturchilari ham ishlatsa, bu yanada foydali bo'lmaydimi?" Shunday qilib, yaxshi yozilgan va foydali funksiyalarni hammaga ochiq tarzda ishlatish mumkin bo'lgan tizim yaratildi, bu tizim *kutubxona* (yoki paket) deb ataladi. Dasturchilar ayniqsa bunday narsalarga juda qiziqishadi. Chunki ular asosan yuqori samaradorlikka intiluvchilardir; birinchi, alohida funksiyalarni bir joyga to'playdilar, keyin ularni yana kattaroq bir tuzilma ostida birlashtiradilar. Buni grafik ko'rinishda tasvirlasak, quyidagicha ko'rinadi.



Kutubxona (library) tushunchasini tushuntirish uchun osonroq va mosroq taqqoslash haqida o'ylaganimda, uning nomi bejiz tanlanmagan bo'lsa kerak. Chunki bu aynan kitoblar bilan to'lgan kutubxonaga o'xshaydi. Kutubxonada ayrim alohida kitoblar mavjud (bularni modul deb tasavvur qilish mumkin), ba'zan esa ketma-ketlikdagi kitoblar to'plamlari ham uchraydi (buni paket deb qarash mumkin). Kitobning ichida esa mazmunni tashkil etuvchi boblar, bo'limlar, matnlar va jumlarlar bor. Bu tuzilmalarni klasslar, funksiyalar va o'zgaruvchilar bilan taqqoslash ham o'rinli.

Hozircha, kutubxona ichida biz uchun oldindan boshqalar tomonidan ishlab chiqilgan foydali kod to'plamlari (asosan funksiyalar) bor, biz esa ularni olib, kodlash jarayonida foydalanamiz, deb bilib qo'ysangiz yetarli.

12 Nihoyat, biz **5-qator**ga yetdik. Buni kod tahlilining oxiri deb xisoblashingiz mumkin. Buning sababi shundaki, 5 va 6 qatorlar alohida kodlar emas, balki bitta ma'noga ega bo'lgan kodlar to'plamidir.


```
def when_start():  
    Entry.print("Hello World!")
```

def when_start(): Bu kod nimani anglatadi? Bu avval bir necha marta aytib o'tilgan funktsiyani bevosita yaratuvchi koddir. Biz boshidanoq funktsiyani ishlatish uchun uni tashqaridan chaqirishni o'rgandik va endi uni o'zimiz yaratishni ham o'rganyapmiz! Matnli kodlashda funktsiya shu qadar muhim. Birinchidan, Python matnli dasturlashda funktsiya yaratish uchun umumiy grammatikani o'rganamiz.

Funktsiyani qanday yaratish (e'lon qilish)

Funktsiya

□ Sintaksis

- E'lon qilish

```
def funksiya_nomi(parametrlar):  
    bajaradigan buyruqlarning to'plami
```

4-ta bo'sh joy yoki 1-ta Tab

- Ishlatilishi

```
funksiya_nomi(argumentlar)
```

Yuqoridagi standart sintaksisning har bir so'z va belgilarining ma'nosini tushunib chiqaylik. *Funktsiya yaratish uchun har doim qator boshida def ("definition"ning qisqartmasi) kalit so'zi bilan boshlash kerak.* Bu so'z "endi yozadigan narsalarim funktsiya yaratish(e'lon qilish) jarayoni" degan ma'noni bildiradi. Keyin, yaratmoqchi bo'lgan funktsiyaga ma'noli va o'zingizga qulay nom berish kerak. Shundan so'ng, funktsiya chaqirilganda chaqiruvchidan olinadigan qiymatlar (bular dasturlash dunyosida "parametr" deb ataladi) qavs ichida vergul (,) bilan ajratilgan holda yoziladi. Va nihoyat, (:) belgisi bilan funktsiya qobig'i tugallanganini ko'rsatib, keyingi qatordan funktsiya ichida bajarilishi kerak bo'lgan haqiqiy kodlar yozilishi boshlanishi kerakligini bildiradi.

Funktsiyalarda parametr va argument o'rtasidagi farqlar

Parametr

- Parametrlar funktsiyani e'lon qilishda foydalaniladigan o'zgaruvchilardir.
- Funktsiyani e'lon qilishda qavs ichida keltirilgan o'zgaruvchilardir.

Argument

- Argumentlar funktsiyani chaqirishda qabul qilingan qiymatlardir.
- Funktsiyani chaqirishda qavs ichida berilgan haqiqiy qiymatlardir.

def when_start(): yuqoridagi standart sintaksisga yana bir bor murojaat qilib tushuntiradigan bo'lsak, bu `when_start` nomli funksiya yaratilayotgani va bu funksiya hech qanday qiymat (parametr) qabul qilmasligini (shuning uchun qavs ichida hech narsa yo'q) bildiradi.

¹²₃₄ Endi oxirgi, ya'ni **6-qatorga** o'tamiz. Bu yerda funksiya chaqirilganda bajarilishi kerak bo'lgan kod yoziladi. Qiziq tomoni shundaki, boshqa qatorlardan farqli o'laroq, bu qator boshlanishidan oldin ma'lum miqdorda bo'sh joy qoldiriladi (bu bo'shliqni **Tab** tugmasini bir marta bosish yoki **Probel(spacebar)** tugmasini to'rt marta bosib hosil qilish mumkin). Dasturlash dunyosida bo'shliq ham ma'noga ega ekanligi haqida ilgari gaplashgan edik. Bu xuddi shunday holatlardan biri bo'lib, Python standart sintaksisiga ko'ra bo'shliq maqsadli ishlatiladi. Bu nafaqat funksiyalarda, balki boshqa ko'plab kodlarda ham qo'llaniladi. Shu sababli, boshidanoq uni yaxshilab o'rganib olish muhimdir. **Bu bo'shliqning ma'nosini oddiy qilib tushuntirsak: har bir qatorda bir xil miqdorda bo'sh joy qoldirilishi bilan, o'sha qatorlar bir kod to'plamiga tegishli ekanligini ko'rsatish uchun ishlatiladi.**

Buni bizning kodimizga taqqoslaydigan bo'lsak, **when_start** funksiyasiga tegishli bo'lgan kodning qayerdan qayergacha ekanligini kompyuterga bildirish uchun bo'sh joy ishlatilgan. Hozir bu funksiya chaqirilganda faqat bitta qator kodni bajaradigan juda oddiy funksiya bo'lgani sababli bu gapni tushunish qiyin bo'lishi mumkin. Lekin kelajakda ko'proq kodlarni bajaradigan va bajarish tarkibi uzun bo'lgan funksiyalarni ko'rganingizda, bu bo'shliqning ma'nosi yanada yaqqolroq bo'ladi.

Endi **Entry.print("Hello World!")** kodini tahlil qilamiz. Uni qismlarga ajratsak: **Entry** — bu kutubxona nomi, undan keyin nuqta (.) qo'yilgan, so'ngra esa **print("Hello World!")** yozilgan. Oldin aytganimizdek, dasturlashda nuqta (.) ham o'ziga xos ma'noga ega. Agar kutubxona nomidan keyin nuqta qo'yilsa, bu o'sha kutubxona ichidagi narsalarni (masalan, funksiyalar, klasslar, o'zgaruvchilar va boshqalar) ishlatishni anglatadi. Bu holda **print("Hello World!")** — **Entry** kutubxonasi ichida joylashgan funksiyadir. Shu sababli, 1-qatorda import orqali olib kelingan **Entry** kutubxonasidagi **print** funksiyasini chaqirayotganimizni bildiradi. **print** nomidan xulosa qilganimizdek, u ekranga ma'lum bir qiymatni chiqarish uchun ishlatiladi. Ekranga chiqariladigan mazmun esa, " (qo'shtirnoq) ichida yoziladi. Bu qo'shtirnoqlar ekranda ko'rsatiladigan matnning boshlanishi va tugashini aniqlab beradi. Masalan, bu kodda "Hello World!" ekranga chiqariladi.

Xo'sh~ mana shunday qilib 6 qatorlik juda oddiy dastur kodi tahlili nihoyasiga yetdi! O'zingizni yengil his qilyapsizmi? Endi, agar siz **Entry** da "Boshlash" tugmasini bossangiz, o'sha 6 qatorlik kod ishga tushadi va ekranda biz xohlagan natija, ya'ni **"Hello World!"** yozuvi paydo bo'ladi. Hammasi yaxshi ishlaydi. Kod to'liq tushuntirildi va muvaffaqiyatli bajarildi. Shunga qaramay, sizda nimadir yechimsiz qolayotgandek yoki qandaydir javobsiz savollar qolayotgandek tuyilishi kerak. Sizda bu his qoldimi? Agar qolmagan bo'lsa ham, xafa bo'lmang.

Qolgan savol shundan iboratki, bizning shu 6 qatorlik kodimizda qanday qilib istalgan natijani olish mumkin? Aslida, natija chiqishi uchun kod to'liq emas. Albatta, kodning boshqa qismi bo'lishi kerak edi. Biz **when_start** funksiyasini faqat e'lon qildik, ammo uni ishlatish uchun chaqirgan kodni hech qayerda ko'rmayapmiz. Ya'ni, **when_start** funksiyasini kodingizda bir joyda chaqirishingiz kerak va faqat shunday qilib, chaqirilgan funksiyaning bajarilishi bilan ekranda chiqishi amalga oshadi. Biroq, bizning 6 qatorlik

dasturda hech qayerda **when_start** funksiyasini bevosita chaqiradigan kod yo'q. Shunda, bu funksiyani kim va qanday chaqirib, qanday qilib ishga tushirdi?

Kolbek (callback) funksiyasi nima?

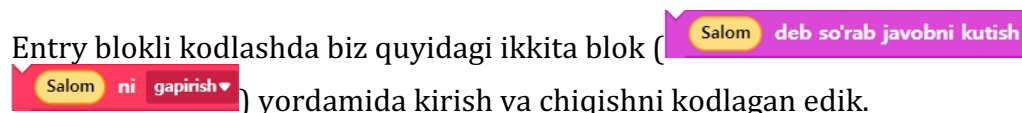
Javobni aytadigan bo'lsak, bizning dasturimizdagi funksiyani to'g'ridan-to'g'ri chaqirmagan bo'lsak ham, bizning dasturimizdan tashqarida, ya'ni tashqi tomondan ushbu funksiya chaqirilgan. Demak, kim chaqirdi? Ha, chaqiruvchi Entry dasturini o'zidir. Entry bizning **when_start** funksiyamizni avtomatik ravishda chaqirgan. Bu yerda umumiy ma'lumot sifatida bilish kerak bo'lgan bir atama mavjud. Agar funksiya bo'lsayu, lekin uni men bevosita chaqirmasdan, tashqi tizim tomonidan biror vaqtda avtomatik tarzda chaqirilsa, bunday funksiyani kolbek (callback - so'zma so'z tarjimai "qayta chaqirish") funksiyasi deb ataladi. Ha, **when_start** kolbek funksiyasi edi. Bunday kolbek funksiyalarining nomini biz o'zimiz xohlagancha belgilay olmaymiz. Entry o'zi oldindan belgilab qo'ygan funksiya nomlarini ishlatishimiz kerak. Ya'ni, Entry ichida qandaydir kelishuv bor, ya'ni foydalanuvchi "Boshlash" tugmasini bosgan zahoti, Entry ichida doimo **when_start** funksiyasini chaqirishi uchun dasturlangan. Agar bu gapga ishonmasangiz, **when_start** nomini atayin o'zgartirib, "Boshla" tugmasini bosing. Kutgan xatti-harakat amalga oshadimi? Biz kutgandek ishlamaydi. Chunki bizning dasturimizda oldindan belgilangan va mavjud bo'lishi kerak bo'lgan o'sha **when_start** funksiyasi yo'q.

Faqatgina 6 qatorlik kichik dastur bo'lsada, matnli kodlashning ko'plab asosiy jihatlarini tushunish uchun yaxshi misol ekanligi shubhasiz. Keyingi bosqichda dasturlashning asosiy elementlarini (ketma-ketlik, takrorlash, shartlar va boshqalar) birma-bir o'rganib chiqamiz.

3.2 Kiritish/Chiqarish (Input/Output)

Dasturda kiritish/chiqarish nima? Bu foydalanuvchidan kerakli ma'lumotlarni olish yoki aksincha, ko'rsatish orqali foydalanuvchi bilan interaktiv aloqa vositasi.

Entry blokli kodlashda biz quyidagi ikkita blok (



) yordamida kirish va chiqishni kodlagan edik.

Endi buni matnli dasturlash orqali amalga oshiraylik. Kiritish/chiqarish funksiyalarini biz avvalgi bo'limda o'rgangan **Entry** kutubxonasidagi ikki turdagi funktsiyani (`print`, `input`) chaqirish orqali bajarish mumkin. Funktsiya nomlari juda intuitiv bo'lib, ularning vazifasi darhol tushunarli. Mazkur funktsiyalar **Entry-Python** dasturida asl Python dan bir oz farq bilan ishlaydi (*asl Python da bu funktsiyalarni kutubxonani chaqirmasdan, to'g'ridan-to'g'ri ishlatish mumkin; bunday funktsiyalar tilning o'ziga xos bo'lgan ichki funktsiyalari bo'lib, "ichki funktsiyalar" deb ataladi*). Ammo **Entry** da ular alohida kutubxona tarkibiga kiradi. Bu funktsiyalarni qanday chaqirishni avvalgi bo'limlarda ko'rib chiqqanmiz: **KutubxonaNomi.FunksiyaNomi** ko'rinishidagi sintaksis yordamida chaqiriladi. Shunday ekan, **Entry.print()** va **Entry.input()** funktsiyalarini chaqirish mumkin. Biroq, funktsiyaning to'g'ri ishlashi uchun chaqiruvchi tomonidan uzatilishi lozim bo'lgan majburiy qiymatlar (parametrlar) qanday ekanini aniq bilish kerak.

Aslida, buni intuitiv tarzda taxmin qilish mumkin: **print** (chiqarish) funktsiyasida foydalanuvchiga ko'rsatmoqchi bo'lgan xabarni uzatish kerak, **input** (kiritish) funktsiyasida esa foydalanuvchidan olishni xohlagan ma'lumotni tushuntiruvchi xabarni uzatish kerak. Ushbu xabarlar odatda istalgan matn bo'lib, kodlash dunyosida bu ma'lumot turi **string** (satrlar qiymatlar) deb ataladi. Shu sababli, xabarni ifodalash uchun matnni " " (**qo'shtirnoqlar**) orasiga yozish kifoya. (Bittalik qo'shtirnoqdan(' ') foydalanish ham mumkin, lekin **Entry-Python** avtomatik ravishda uni ikkitalik qo'shtirnoqqa o'zgartiradi.) Bundan tashqari, chiqarilmoqchi bo'lgan xabarni bir nechta qismga ajratib, ularni birlashtirib chiqarish ham qiziqarli usuldir. Bir nechta matn qismlarini birlashtirish uchun foydalaniladigan operator **+** bo'lib, u "matnlarni birlashtirish" degan mantiqiy ma'noni beradi.

Keling, hozirgacha nazariy jihatdan o'rgangan barcha narsalarni birlashtiramiz va foydalanuvchining yoshini kiritish va uni chiqaradigan matnli dasturlash yordamida juda oddiy dasturni kodlaymiz.

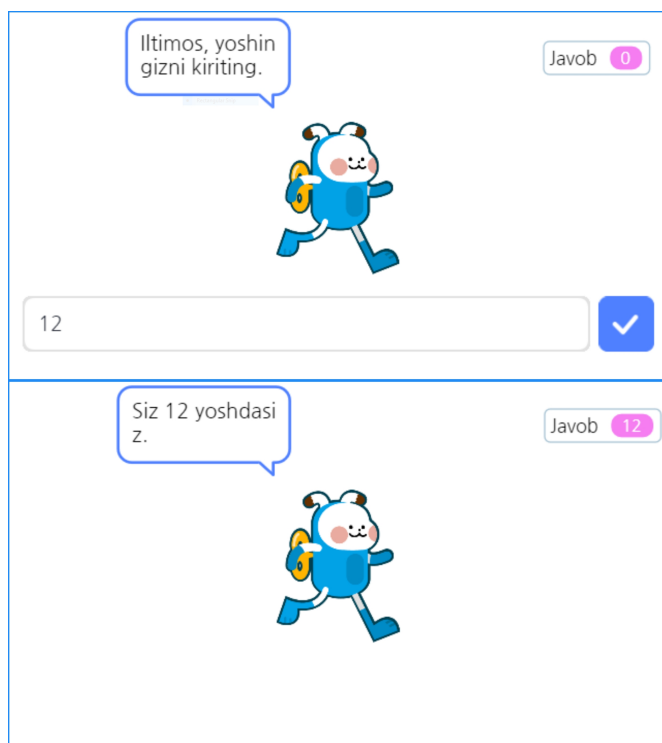
Entry-Python

```
1. # (1)Entrybot's Python code
2.
3. import Entry
4.
5. def when_start():
6.     Entry.input("Iltimos, yoshingizni kiriting.")
7.     Entry.print("Siz " + Entry.answer() + " yoshdasiz.")
```

Blokli kodlash



Ijro natijasi



Kodlarni tushunish qiyin emasmi? Biroz notanish bo'lgan joy **Entry.answer()** deb yozilgan qismidir. Bu funksiya nima uchun kerak? *Aslida, bu funksiya faqat **Entry-Python** uchun zarur (haqiqiy Pythonda foydalanuvchi kiritgan qiymatni dasturchi o'zi aniqlagan o'zgaruvchiga saqlab ishlatadi, shuning uchun bunday maqsadga mo'ljallangan funksiya kerak emas).* Ushbu funksiyaning vazifasi foydalanuvchi tomonidan kiritilgan qiymatni bilib olishdir. Bu yerda **Entry.print** funksiyasi ichida chiqarish xabarini tuzishda, foydalanuvchi kiritgan qiymatni olib uni natija sifatida ko'rsatishda foydalanilgan.

Ichki ishlash jarayonini biroz batafsilroq tushuntirsak, foydalanuvchi qandaydir qiymat kiritganida, bu qiymatini kodning boshqa qismida qayta ishlatish uchun uni qandaydir joyga vaqtinchalik saqlash kerak bo'ladi (odatda bunday saqlash joyi **o'zgaruvchi** deb ataladi). Keyin shu o'zgaruvchi orqali kiritilgan qiymati kodning boshqa qismlarida ishlatiladi. Ammo bizning kodimizda o'zimiz yaratgan birorta o'zgaruvchi bo'lmasada, kerakli natijani amalga oshirdi. Bu qanday sodir bo'ldi? Haqiqatda, **Entry** ichki tizimida foydalanuvchi kiritgan qiymati vaqtinchalik yashirin o'zgaruvchida saqlanadi. Bizga esa faqat ushbu qiymatini o'qib olish uchun **answer()** funksiyasi taqdim etiladi. Yuqorida

aytib o'tilganidek, haqiqiy Python dasturlashida bunday usul ishlatilmaydi. Ammo **Entry** blokli kodlashda foydalanuvchilar uchun maksimal qulaylikni ta'minlash maqsadida bu zarurat tug'ilgan. **Entry-Python** ham blokli kodlashni to'liq davom ettiruvchi va foydalanuvchilarni matnli dasturlashga o'tishga o'rgatuvchi oraliq ko'priq sifatida xizmat qiladi. Shuning uchun ham, **Entry-Python** Pythonni mukammal o'rganish vositasi emas, balki blokli kodlashdan matnli dasturlashga o'tishda yordam beruvchi vosita ekanligini ushbu misoldan ham bilib olishimiz mumkin.

3.3 O'zgaruvchi (Variable)

Biz Entry blokli kodlashda o'zgaruvchi va ro'yxat deb nomlangan elementlarni ishlatib ko'rganmiz. Ikkalasi ham ma'lumotlarni saqlash uchun mo'ljallangan joy bo'lib xizmat qiladi. Avval o'zgaruvchining nima ekanligini va uning maqsadi nimadan iboratligini eslab olaylik. O'zgaruvchi — nomidan ma'lum bo'lgandek, o'zgaruvchi qiymatlarni vaqtinchalik saqlash uchun ajratilgan joy. Blokli kodlashda biz ba'zi bloklarni qo'llaganimizda, ko'pincha oldindan belgilangan biror amalni bajarish uchun muayyan qiymatlar aniqlab berilganini ko'ramiz. Ammo, oldindan belgilab bo'lmaydigan qiymatlar (masalan, foydalanuvchi tomonidan kiritilgan tasodifiy qiymatlar) asosida ishlaydigan dasturlar uchun o'zgaruvchilardan foydalanish zarur bo'ladi.

Yuqorida aytib o'tilgan misolga o'xshash holda, foydalanuvchi tomonidan tasodifiy kiritilgan ikki raqamni qabul qilib, ularning yig'indisini hisoblashga qaratilgan oddiy qo'shish kalkulyatori dasturini o'zgaruvchidan foydalanib yozamiz.

Entry-Python

```
1. # (1)Entrybot's Python code
2.
3. import Entry
4.
5. first = 0
6. second = 0
7. result = 0
8.
9. def when_start():
10.     Entry.print("Bu siz kiritgan ikkita raqamni qo'shadigan dastur.")
11.     Entry.wait_for_sec(2)
12.
13.     Entry.input("Iltimos, birinchi raqamni kiriting.")
14.     first = Entry.answer()
15.     Entry.input("Ikkinchi raqamni kiriting.")
16.     second = Entry.answer()
17.
18.     result = (first + second)
19.     Entry.print(("Kiritilgan ikkita raqamning yig'indisi " + result) + " ga teng.")
```

Blokli kodlash



Ijro natijasi

Bu siz kiritgan ikkita raqamni qo'shadigan dastur.

Javob 0

first 0

second 0

result 0

Iltimos, birinchi raqamni kiring.

Javob 0

first 0

second 0

12

✓

Ikkinchi raqamni kiring.

Javob 12

first 12

second 0

13

✓

Kiritilgan ikkita raqamning yig'indisi 25 ga teng.

Javob 13

first 12

second 13

result 25

Biz hali matn dasturlash bilan yaxshi tanish bo'lmaganimiz sababli, kod miqdori to'satdan ko'payib ketgandek tuyulishi mumkin, lekin aslida bu o'qishni oshirish uchun ataylab bo'sh qatorlar bilan kod bo'laklariga ajratilgan, shuning uchun qatorlar soni katta tuyulishi mumkin, agar siz uni diqqat bilan izohlasangiz, unda murakkab tarkib deyarli yo'qligini ko'rasiz.

Hozirgacha o'rganilmagan, birinchi marta uchrayotgan qism — bu 5~7-qatorlarda yozilgan 3 ta kod qatoridir. Ushbu qatorlarda **first**, **second**, va **result** deb nomlangan 3 ta o'zgaruvchi e'lon qilinadi va ushbu o'zgaruvchilarga dastlabki qiymat sifatida 0 belgilanadi. **Python tilida o'zgaruvchi yaratish sintaksisi quyidagicha:**

o'zgaruvchi nomi = boshlang'ich qiymat

*Bu yerda o'zgaruvchi nomini kerakli, tasodifiy bir nom bilan belgilash mumkin, lekin maqsadga muvofiq, kodni o'qishda uning vazifasini osongina tushunish uchun mazmunli nom tanlash tavsiya etiladi. Yuqoridagi misolda foydalanuvchining birinchi kiritgan qiymati (first), ikkinchi kiritgan qiymati (second), va ushbu ikkita qiymatning yig'indisi (result) ma'nosini ifodalash uchun ushbu o'zgaruvchi nomlari ataylab ishlatilgan. O'zgaruvchilarga dastlab yaratilayotganda biron-bir tasodifiy qiymat (bunga "boshlang'ich qiymat" deyiladi) berilib, qiymatga ega holda ishga tushiriladi. Odatda, `_0_` dastlabki qiymat sifatida ishlatiladi. Ushbu sintaksisda e'tibor qaratish kerak bo'lgan bir narsa bor: **'=' belgisining ma'nosi matematikadagi "teng" degan ma'noni emas, balki, '=' belgisining o'ng tomonidagi qiymatni chap tomondagi o'zgaruvchiga tayinlash (sozlash) degan ma'noni anglatadi. Buni albatta yodda saqlash lozim.***

12 **34** Shunday qilib, 5-qatordagi **first = 0** kodi, '=' belgining chap tomonida joylashgan **first** o'zgaruvchisiga '=' belgining o'ng tomonida joylashgan **0** qiymatini saqlashni anglatadi.

O'zgaruvchi nomini ixtiyoriy tarzda ishlatish mumkin, deb aytilgan bo'lsada, aslida Python dasturlash tilida o'zgaruvchiga nom berishda rioya qilinishi kerak bo'lgan 2 asosiy qoida mavjud:

- O'zgaruvchi nomi raqam bilan boshlanishi mumkin emas.
- O'zgaruvchi nomi orasida bo'sh joy bo'lishi mumkin emas.

Bu yerda, matematik ma'noda chap va o'ng tomondagi qiymatlar bir-biriga tengmi yoki yo'qligini kodda ifodalash uchun qanday belgi ishlatish kerakligi qiziq bo'lishi mumkin. Bu holatda **==** belgisi ishlatiladi. **Ya'ni, '=' belgisini bitta emas, ketma-ket ikki marta ishlatish orqali chap va o'ng tomondagi qiymatlarning bir-biriga tengligini solishtirish ma'nosini anglatadi.**

12 **34** 10~11-qatordagi kodlar to'plami dasturda foydalanuvchiga dasturimizning qanday dastur ekanligini tushuntirish uchun foydalaniladi. 10-qatordagi chiqarish funksiyasi ilgari ishlatilgan, 11-qatordagi **Entry.wait_for_sec(2)** kodi esa birinchi marta uchraydi. Uning ma'nosini tushunish qiyin emas: bu Entry kutubxonasidagi **wait_for_sec** funksiyasini chaqirishdir. Ushbu funksiya blok kodlashdagi "~ soniya kutish" blokiga o'xshaydi. Kutish davomiyligini funksiyaga qiymat sifatida uzatish kerak, va bu yerda 10-qatorda chiqarilgan matnni ekranda 2 soniya davomida ko'rsatib turish uchun 2 qiymati berilgan.

12 **34** 13~16-qatordagi kodlar to'plami foydalanuvchidan ketma-ket ikkita tasodifiy son kiritishni so'raydi va ularni mos ravishda **first** va **second** o'zgaruvchilariga saqlaydi.

Nima uchun foydalanuvchi kiritgan qiymatni `Entry.answer()` orqali o'qib, saqlash kerakligi haqida avvalgi bobda [kiritish-chiqarish funksiyalari](#) qismida tushuntirilgan.

 18-qatorda esa **first** va **second** o'zgaruvchilarida saqlangan qiymatlar o'qilib, ularning yig'indisi uchinchi o'zgaruvchi, ya'ni **result** ga saqlanadi.

 19-qator kodda foydalanuvchi kiritgan ikki sonning yig'indisi saqlangan o'zgaruvchidagi qiymatni foydalanuvchiga ko'rsatish orqali dastur o'z maqsadiga erishadi va yakunlanadi.

Agar biz blokli kodlashda o'zgaruvchilarni ishlatgan bo'lsak, ularning ma'nosi va qo'llanilishini yaxshi bilamiz. Matnli dasturlashga o'tilganda ham bu ma'no va maqsad umuman o'zgarmaydi. Faqatgina matnli dasturlashda (bu yerda Python) foydalanilganda, shu tilning o'zgaruvchi ishlatish sintaksisiga amal qilib, foydalanish kifoya.

3.4 Shart (Condition)

Blok kodlashda juda ko'p ishlatiladigan ikki xil blok mavjud: **“Agar ~ bo'lsa”** va **“Agar ~ bo'lsa, bo'lmasa ~”**. Ularning vazifasi shundaki, bizning kodimiz ketma-ket bajarilayotganida, ma'lum bir shartga qarab, ba'zi harakatlarni bajarish yoki bajarmaslikni ta'minlashdir.

Oldingi bobda yaratilgan kalkulyator dasturida foydalanuvchidan kiritilgan ikkita tasodifiy qiymatni faqat qo'shib berishdan tashqari, foydalanuvchiga yaxshiroq foydalanish imkoniyatini yaratish uchun, faqat qo'shish emas, balki ayirishni ham amalga oshirish va qaysi amalni bajarishni tanlash imkoniyatini foydalanuvchiga taqdim etuvchi yaxshiroq dasturga rivojlantiraylik. Shuni yodda tutish kerakki, dasturiy ta'minot yaratish bir martalik jarayon bilan tugamaydi. U doimiy ravishda rivojlanib, yaxshilanib, yangi versiyalarni chiqara boradi. Bu jarayon dasturiy ta'minotning hayot sikli (software lifecycle) deb ataladi.

Entry-Python(1)

```
1. # (1)Entrybot's Python code
2.
3. import Entry
4.
5. first = 0
6. second = 0
7.
8. def when_start():
9.     Entry.print("Bu siz kiritgan ikkita raqamni qo'shadigan yoki ayiradigan
dastur.")
10.    Entry.wait_for_sec(3)
11.
12.    Entry.input("Iltimos, birinchi raqamni kiriting.")
13.    first = Entry.answer()
14.    Entry.input("Ikkinchi raqamni kiriting.")
15.    second = Entry.answer()
16.
17.    Entry.input("Agar siz qo'shmoqchi bo'lsangiz, '+' kiriting va agar siz
ayirmoqchi bo'lsangiz, '-' kiriting.")
18.    if Entry.answer() == "+":
19.        Entry.print("Kiritilgan ikkita raqamning yig'indisi " + (first +
second) + " ga teng")
20.    if Entry.answer() == "-":
21.        Entry.print("Kiritilgan ikkita raqamning ayirmasi " + (first -
second)) + " ga teng")
```

Blokli kodlash(1)



Entry-Python(2)

```
1. # (1)Entrybot's Python code
2.
3. import Entry
4.
5. first = 0
6. second = 0
7.
8. def when_start():
9.     Entry.print("Bu siz kiritgan ikkita raqamni qo'shadigan yoki ayiradigan
dastur.")
10.    Entry.wait_for_sec(3)
11.
12.    Entry.input("Iltimos, birinchi raqamni kiriting.")
13.    first = Entry.answer()
14.    Entry.input("Ikkinchi raqamni kiriting.")
15.    second = Entry.answer()
16.
17.    Entry.input("Agar siz qo'shmoqchi bo'lsangiz, '+' kiriting va agar siz
ayirmoqchi bo'lsangiz, '-' kiriting.")
18.    if Entry.answer() == "+":
19.        Entry.print("Kiritilgan ikkita raqamning yig'indisi " + (first + second)) +
" ga teng")
20.    else:
21.        if Entry.answer() == "-":
22.            Entry.print("Kiritilgan ikkita raqamning ayirmasi " + (first - second))
+ "ga teng")
```

Blokli kodlash(2)



Ijro natijasi



Ikkinchi raqamni kiring.

Javob 13

first 13

second 0

12

✓

Agar siz qo'shm oqchi bo'lsangiz, '+' kiriting va agar siz ayirmoqchi bo'lsangiz, '-' kiriting.

Javob 12

first 13

second 12

-

✓

Kiritilgan ikkita raqamning ayirmasi 1 ga teng

Javob -

first 13

second 12

Agar avvalgi dasturda 3-ta o'zgaruvchi (first, second, result) ishlatilgan bo'lsa, **bu safar o'zgaruvchilar soni 2-ta (first, second) ga qisqartirildi**. Qisqartirish usuli shundan iboratki, avvalgi dasturda ikki qiymatning hisoblangan natijasi result o'zgaruvchisiga saqlanib, keyin shu o'zgaruvchidagi qiymat chiqarilgan bo'lsa, bu safar hisoblash natijasi o'zgaruvchi ishlatmasdan, bevosita **(first + second)** yoki **(first - second)** ko'rinishida hisoblash ifodasini qavslar ichida yozilib, chiqarish funksiyasi (print funksiyasi) ichida to'g'ridan-to'g'ri hisoblash va natijani chiqarish amalga oshirildi.

¹²₃₄ 17-qatorni ko'rib chiqaylik. Foydalanuvchiga '+' yoki '-' belgilaridan birini tanlash va kiritish imkoniyati berilganligi sababli, foydalanuvchi ushbu amallardan birini to'g'ri kiritadi, deb faraz qilinadi. Foydalanuvchi tanlagan amal turiga qarab, kodni boshqacha bajarish talab qilinadi. Shu sababdan **shart operatori** ishlatilgan. Python tilida shart operatorini ishlatish sintaksisi quyidagicha bo'ladi:

if shart:

└─ shart qoniqqan(yoki rost) bo'lsa nima qilish kerak
4-ta bo'sh joy yoki 1-ta TAB

¹²₃₄ Keling, yuqorida keltirilgan sintaksisga asoslanib 18-19 qatorlardagi kodni tushuna olamizmi.

```
1. if Entry.answer() == "+":  
2.     Entry.print("Kiritilgan ikkita raqamning yig'indisi " + (first + second) + " ga  
teng")
```

Oldingi bob da '==' belgisining (operatorining) ma'nosi tushuntirilgan edi. U chap va o'ng tomondagi ikkita qiymatning tengligini tekshirish uchun ishlatiladigan operator bo'lib, bu yerda foydalanuvchi kiritgan qiymatning '+' ekanligini aniqlash uchun ishlatilmoqda. Agar foydalanuvchi kiritgan qiymat '+' bo'lsa, **if** bilan boshlangan qatordan keyingi kodlar bajariladi. Bu yerda kod yozishda e'tibor berilishi kerak bo'lgan jihat — **biz oldingi Hello World misoli** ning tahlilida ko'rganimizdek, **if shartidan keyin ataylab bo'shliq (odatda klaviaturada bo'sh joy(spacebar) tugmasi bilan 4 marta yoki tab tugmasi bilan 1 marta bosish orqali yaratiladi) qoldirish lozim. Har bir qatorning bir xil hajmdagi bo'shliq bilan boshlanishi, if sharti bajarilgan holda ishga tushiriladigan kodlar to'plamining boshlanishi va tugash joylarini aniqlash imkonini beradi.** Shuning uchun, bo'shliqlarni to'g'ri va aniq kiritishni yaxshi tushunib olish kerak.

¹²₃₄ Yuqorida shart operatorining ishlatilishi to'liq tushunarli bo'lsa, keyingi 20~21-qator kodlarni tushunish juda oson bo'ladi.

Oxirgi masala sifatida, yuqoridagi misol kodni blokli kodlash (1 va 2) va matnli dasturlash (1 va 2) ga ajratganimizning sababini tushuntirish lozim. 1 va 2 turdagi kodlar bir xil vazifani bajaradi, ammo ularning amalga oshirilish usulida ozgina farq bor. Shu bilan birga, bitta qo'shimcha shart operatori sintaksisini tushuntirish maqsadi bor edi.

Agar biz blokli kodlashdagi "Agar ~ bo'lsa" blokining matnli kodlashda qanday ifodalanishini o'rgangan bo'lsak, endi "Agar ~ bo'lsa, bo'lmasa ~" blokining matnli dasturlashda ham mavjud ekanligini taxmin qilishimiz mumkin. Buni ishlatish sintaksisi quyidagicha bo'ladi:

if shart:

└─ shart qoniqqan(yoki rost) bo'lsa nima qilish kerak
4-ta bo'sh joy yoki 1-ta TAB

else :

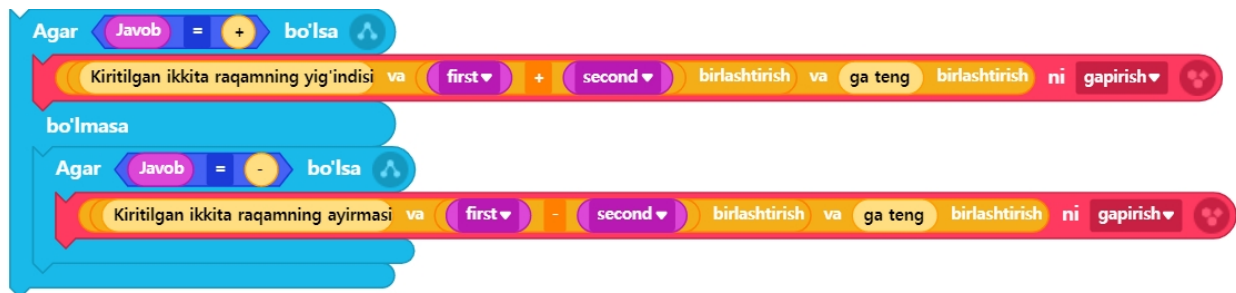
└─ shart qoniqmagan(yoki yolg'on) bo'lsa nima qilish kerak
4-ta bo'sh joy yoki 1-ta TAB

```

1. if Entry.answer() == "+":
2.     Entry.print("Kiritilgan ikkita raqamning yig'indisi " + (first + second) + " ga teng")
3. else:
4.     if Entry.answer() == "-":
5.         Entry.print("Kiritilgan ikkita raqamning ayirmasi " + (first - second) + " ga
teng")

```

Ushbu kodni quyidagicha ko'rsatilgan blokli kodlash bilan taqqoslash orqali osonroq tushunish mumkin:



Biroq, if~else~ ko'rinishidagi matnli dasturlashda shart operatoridan foydalanishda ham oldingi kabi har bir kod qatorida bir xil bo'shliq uzunligini saqlashga e'tibor qaratish kerak. **Blokli kodlashda shart operatorining boshlanishi va tugash joyi vizual tarzda aniq ko'rsatilgani sababli, uni intuitiv tarzda tushunishimiz oson edi. Ammo matnli dasturlashda buni faqat matn orqali ifodalashda cheklovlar mavjud. Shu sababli, blokli kodlashda yopilgan kodlar to'plami matnli dasturlashda har bir qatorni bir xil miqdorda bo'shliq(identatsiya) bilan to'g'rilash orqali ifodalanishini tushunish lozim.**

3.5 Takrorlash (Loop)

Biz blokli kodlashda kodning oqimini boshqarish maqsadida (shuning uchun Entryning blok kategoriyasida “**oqim**” kategoriyasiga kiradi) eng asosiy bloklardan biri bo‘lgan “**takrorlash**”ni ko‘p marta ishlatganmiz. Takrorlanishi kerak bo‘lgan kod qismini qanday va qancha marta takrorlash kerakligini aniqlash uchun Entryning “oqim” kategoriyasidagi takrorlash bilan bog‘liq bloklar uchta asosiy turga bo‘linadi: “davomiy takrorlash / belgilangan marta takrorlash / ma‘lum bir shart davomida takrorlash”.

Birinchi, “ma‘lum bir shart davomida takrorlash”ni Python sintaksisida ifodalash quyidagicha bo‘ladi:

while *shart*:

 *shart* qoniqqan(yoki rost) bo‘lsa nima qilish kerak

4-ta bo‘sh joy yoki 1-ta TAB

while inglizcha bog‘lovchining o‘zi ham “(biror narsa) **davomida**” degan ma‘noni anglatadi. Shunday qilib, while operatorida yozilgan ‘shart’ qoniqqan davomida uning ostida joylashgan kodlar doimiy ravishda takroran bajariladi. Buni shunday tushunish mumkin: takrorlash amalga oshirilayotgan vaqt davomida har safar ‘shart’ning bajarilayotganini yoki bajarilmayotganini tekshirish kerak. Shu vaqt davomida holat o‘zgarishi va shuning uchun ‘shart’ning natijasi yolg‘on bo‘lib bajarilmay qolishi mumkin. O‘sha vaqtda takrorlash to‘xtatiladi, va while operatoridan chiqiladi. Shundan so‘ng while kod to‘plamidan keyin yozilgan kodlar ketma-ket bajarila boshlaydi.

Biz odatda takrorlash davomida ‘shart’ o‘zgarib, shu bilan birga ‘shart’ning natijasi yolg‘on bo‘lib bajarilmay qolganda, takrorlashdan chiqishni kutamiz. Ammo, agar takrorlashni ataylab cheksiz davom ettirmoqchi bo‘lsak, nima qilish kerak? Buni shunday amalga oshirish mumkin: ‘shart’ni hech qachon tekshirilmay qolmaydigan qilib belgilash. Boshqacha qilib aytganda, ‘shart’ni doim **rost** (True) bo‘lishiga majbur qilish kerak. Buni Python tilidagi matnli dasturlash sintaksisi yordamida quyidagicha ifodalash mumkin:

while True:

 cheksiz bajariladigan tarkib

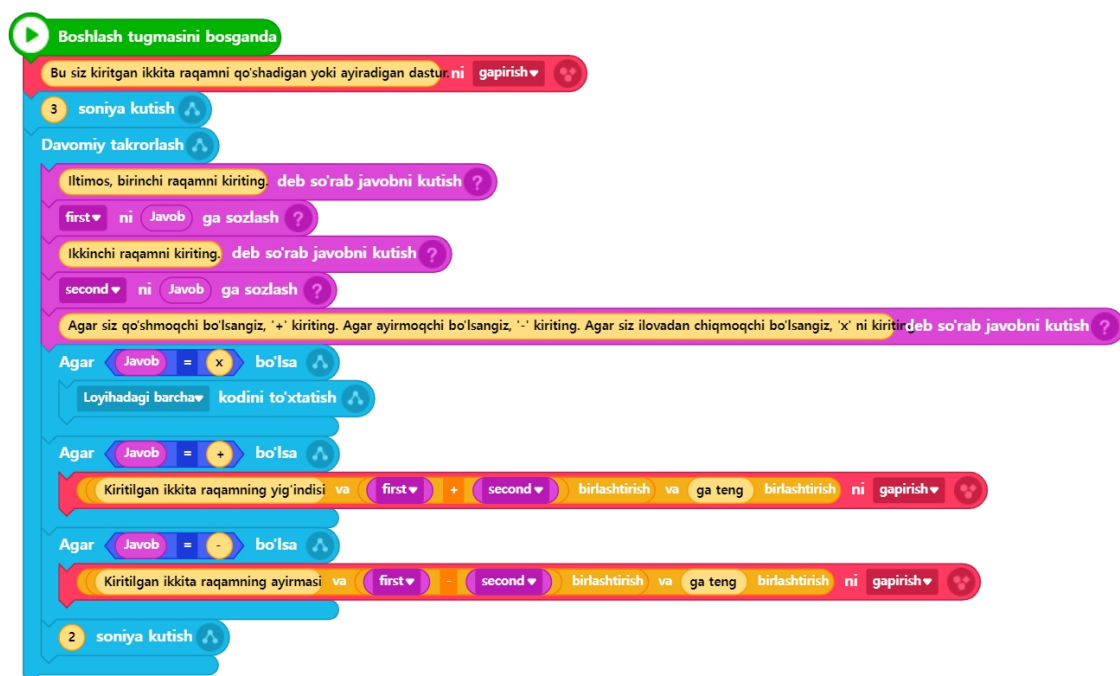
4-ta bo‘sh joy yoki 1-ta TAB

Yuqorida o‘rganilgan texnikadan foydalanib, avvalgi bo‘limlarda yaratilgan kalkulyator dastur(yoki ilova)ini yanada takomillashtirib, foydalanuvchining foydalanish qulayligini oshirishga harakat qilamiz. Foydalanuvchi nuqtai nazaridan, kalkulyator yordamida hisoblash jarayonini faqat bir marta bajarish bilan cheklanmaslik, balki davom ettirish imkoniyati muhim. Hozirgi sharoitda foydalanuvchi hisoblashni qayta-qayta davom ettirmoqchi bo‘lsa, dastur(yoki ilova)ni qayta ishga tushirishdan boshqa yo‘li yo‘q. Bu esa foydalanish qulayligini cheklaydi. Foydalanuvchi istagancha hisoblashni davom ettira olishi va kerak bo‘lmaganida dasturni to‘xtata olishi kerak. Ushbu foydalanuvchi talablariga javob berish uchun, biz kodlarimizga takrorlash operatorini qo‘shishimiz zarur.

Entry-Python

```
1. # (1)Entrybot's Python code
2.
3. import Entry
4.
5. first = 0
6. second = 0
7.
8. def when_start():
9.     Entry.print("Bu siz kiritgan ikkita raqamni qo'shadigan yoki ayiradigan
dastur.")
10.    Entry.wait_for_sec(3)
11.
12.    while True:
13.        Entry.input("Iltimos, birinchi raqamni kiriting.")
14.        first = Entry.answer()
15.        Entry.input("Ikkinchi raqamni kiriting.")
16.        second = Entry.answer()
17.
18.        Entry.input("Agar siz qo'shmoqchi bo'lsangiz, '+' kiriting. Agar ayirmoqchi
bo'lsangiz, '-' kiriting. Agar siz ilovadan chiqmoqchi bo'lsangiz, 'x' ni kiriting.")
19.        if Entry.answer() == "x":
20.            Entry.stop_code("all")
21.        if Entry.answer() == "+":
22.            Entry.print("Kiritilgan ikkita raqamning yig'indisi " + (first + second)
+ " ga teng")
23.        if Entry.answer() == "-":
24.            Entry.print("Kiritilgan ikkita raqamning ayirmasi " + (first - second) +
" ga teng")
25.            Entry.wait_for_sec(2)
```

Blokli kodlash



Ijro natijasi

Agar siz qo'shm oqchi bo'lsangiz, '+' kiriting. Agar ayirmoqchi bo'lsangiz, '-' kiriting. Agar siz ilovada n chiqmoqchi bo'lsangiz, 'x' ni kiriting.



Javob 5

first 3

second 5

x

✓

Endi yozilgan kodni tushunaylik. Takomillashtirilgan kalkulyator ilovasida cheksiz takrorlanishi kerak bo'lgan kodlar **while True:** dan keyingi qatordan boshlab joylashganini va ularning barchasi **identatsiya**(indenting - 4 bo'sh joy yoki 1 tab orqali bo'shliq qoldirish) bilan yozilganini ko'rish mumkin. Foydalanuvchi bizning kalkulyator ilovamizni yopmoqchi bo'lganida, unga ko'rsatmalarda aytilganidek, 'x' qiymatini kiritadi. Ushbu qism kodning 19~20-qatorlarida bajariladi. Agar foydalanuvchi 'x' qiymatini kiritgan bo'lsa, Entry kutubxonasidagi dastur(ilova)ning kodlarni to'xtatish funksiyasi bo'lgan **stop_code** funksiyasi chaqiriladi. Barcha kodlarni to'xtatish uchun esa funksiyaga "all" argumenti yuborilganini ko'rish mumkin.

3.6 Ro'yxat (List)

Blokli kodlashda ro'yxat deb atalgan saqlash joyining maqsadi nima edi? Oldin **o'zgaruvchi faqat bitta o'zgaruvchan qiymatni (yoki oldindan aniq bo'lmagan qiymatni) saqlagan bo'lsa, ro'yxat deb ataladigan saqlash joyi faqat bitta emas, balki bir nechta qiymatlarni saqlash imkonini beradi.**

Python tilida bunday joy ro'yxat (list) deb ataladi, boshqa matnli dasturlash tillarida esa odatda "massiv" (array) deb yuritiladi.

Blokli kodlashdagi ro'yxatni Entry-Python matnli dasturlashida qanday ishlatish mumkin? Misol orqali Entry-Pythonda qanday kod yozish mumkinligini batafsil ko'rib chiqamiz. [3.3-bo'lim](#) da yaratilgan faqat qo'shish funksiyasiga mo'ljallangan kalkulyator dasturidan foydalanib, ro'yxatlar yordamida bu dastur foydalanuvchanligini yanada yaxshilash mumkin. [3.3-bo'lim](#) dagi kalkulyator foydalanuvchidan faqat ikkita tasodifiy qiymatni qabul qilib, ularning yig'indisini hisoblaydi. Endi foydalanuvchi faqat ikkita qiymat emas, balki ko'plab ketma-ket qiymatlarni kiritishi va kiritilgan barcha qiymatlarning umumiy yig'indisini hisoblash imkoniyatiga ega bo'lsa, bu foydalanuvchi uchun yanada qulayroq dastur(ilova)ga aylanadi.

Avval, ro'yxatni qanday ishlatish mumkinligini belgilaydigan sintaksisni ko'rib chiqamiz:

```
ro'yxat_nomi = [matn, raqam va boshqa bir nechta qiymatlar. Qiymatlar  
vergul( , ) bilan ajratiladi]
```

Ro'yxatning nomini xuddi o'zgaruvchilar uchun nom qo'yishda bo'lgani kabi istalgan tarzda belgilash mumkin. Ammo, kod ma'nosini men yoki boshqa kishi tez va samarali tushunishi uchun har doim mazmunli nom berishga odatlanish kerak. Avval ham aytib o'tganimizdek, bu odat dasturlash olamidagi asosiy fazilatlardan biridir.

Ro'yxat nomini istalgancha belgilash mumkin bo'lsada, aslida Python tilida ro'yxatga nom qo'yishda quyidagi ikki muhim qoidaga amal qilish kerak:

- Ro'yxat nomi raqam bilan boshlanishi mumkin emas.
- Ro'yxat nomida bo'sh joy bo'lishi mumkin emas.

Quyida sintaksisga muvofiq yozilgan ro'yxatlarning misollari keltirilgan:

```
words = ["apple", "banana", "grape"]
```

```
numbers = [1, 3, 10, 25, 2]
```

```
complex = ["apple", 2, 32, "berry", 56, 5]
```

words nomidagi ro'yxatda satrli(string) turidagi 3 ta elementlar mavjud.

numbers nomidagi ro'yxatda sonli turidagi 5 ta elementlar mavjud.

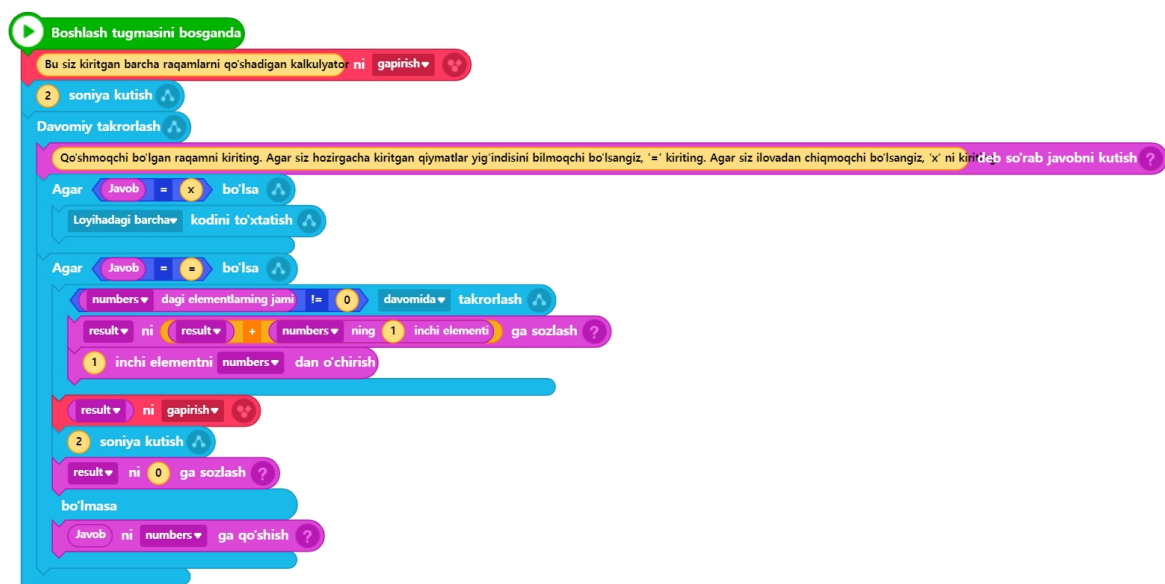
complex nomidagi ro'yxatda satrli va sonli turidagi elementlar aralash holda 6 ta elementlar mavjud.

Hozirgacha biz ro'yxatni aniqlash va bir vaqtning o'zida boshlang'ich qiymatlarni belgilashni o'rgandik. Endi o'rganishimiz kerak bo'lgan narsa – ro'yxat ichida saqlangan qiymatlarga(elementlarga) qanday murojaat qilish va ularni o'qish, ro'yxatga yangi qiymat qo'shish yoki mavjud qiymatlarni o'chirish usullari haqida. Yuqorida tilga olingan ro'yxat bilan ishlash usullari bo'yicha bilimlarni bu bo'limda ko'rib chiqamiz. Bundan tashqari, rivojlangan qo'shish kalkulyatori dasturi uchun yozilgan kodlarni tahlil qilish orqali, ushbu usullarni amalda qanday qo'llashni o'rganamiz.

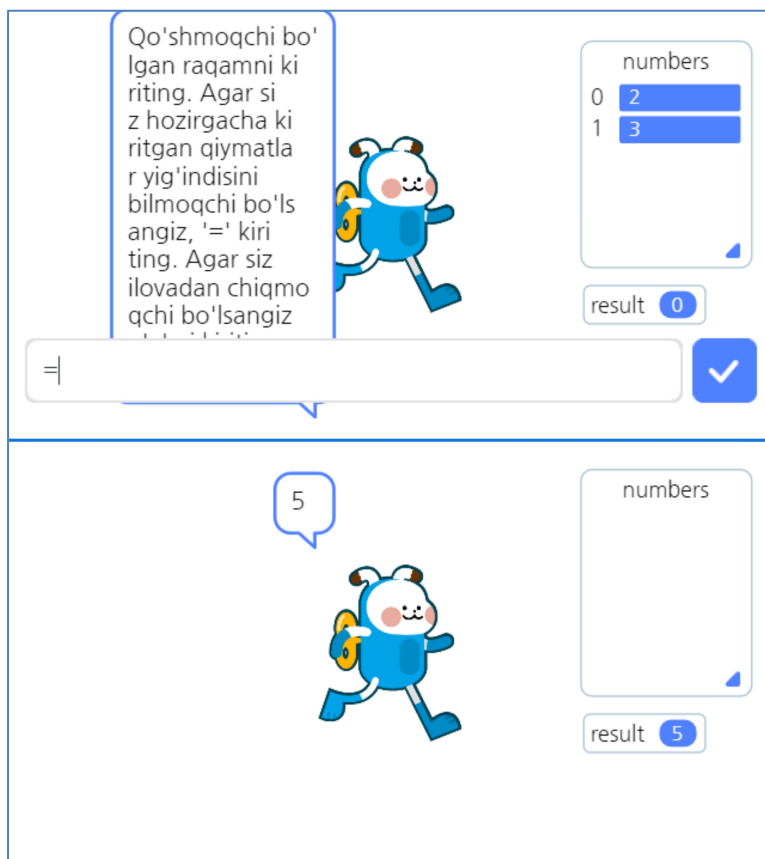
Entry-Python

```
1. # (1)Entrybot's Python code
2. import Entry
3.
4. result = 0
5. numbers = []
6.
7. def when_start():
8.     Entry.print("Bu siz kiritgan barcha raqamlarni qo'shadigan kalkulyator")
9.     Entry.wait_for_sec(2)
10.
11.     while True:
12.         Entry.input("Qo'shmoqchi bo'lgan raqamni kiriting. Agar siz hozirgacha
13. kiritgan qiymatlar yig'indisini bilmoqchi bo'lsangiz, '=' kiriting. Agar siz ilovadan
14. chiqmoqchi bo'lsangiz, 'x' ni kiriting.")
15.
16.         if Entry.answer() == "x":
17.             Entry.stop_code("all")
18.
19.         if Entry.answer() == "=":
20.             while len(numbers) != 0:
21.                 result = result + numbers[0]
22.                 numbers.pop(0)
23.
24.             Entry.print(result)
25.             Entry.wait_for_sec(2)
26.             result = 0
27.         else:
28.             numbers.append(Entry.answer())
```

Blokli kodlash



Ijro natijasi



12 **34** Birinchidan, 5-qatorda **numbers** deb nomlangan ro'yxatning e'lon qilinishi biroz g'ayrioddiy. Ko'rishimiz mumkinki, ro'yxat dastlabki qiymatlarsiz, ya'ni u to'rtburchak qavslar ichida hech qanday qiymatlarsiz e'lon qilingan (shunchaki to'rtburchak qavslar ochilgan va yopilgan). Bu kabi e'lon qilishning sababi mantiqan tushunarli: bizning ilovamiz foydalanuvchi cheklanmagan miqdorda kiritishi mumkin bo'lgan sonlarni saqlash uchun vaqtinchalik joyga muhtoj. Shu maqsadga mos keladigan joy sifatida ro'yxatdan foydalanamiz. Foydalanuvchi hech narsa kiritmagan bo'lsa ham, ro'yxat mavjud bo'lishi kerak, shuning uchun ro'yxatda hech elemetlarsiz bo'sh holatda e'lon qilingan deb taxmin qilish mumkin.

12 **34** Endi 26-qatorga nazar tashlaymiz. 26-qator ro'yxatga foydalanuvchi kiritgan sonli qiymatini yangi element sifatida qo'shish uchun yozilgan kodni ifodalaydi. Bu yerda **append** funksiyasi yordamida foydalanuvchi kiritgan qiymatni **Entry.answer()** orqali o'qib, **numbers** ro'yxatiga qo'shyapmiz. Shu paytda **numbers.append()** deb chaqirilayotgan funksiyaning ro'yxat nomi bilan bog'langan holatda ishlatilayotgani qiziqarli ko'rinadi. Bu qanday qilib ishlayapti? **Sababi shundaki, ro'yxatning o'zi obyekt (Object) hisoblanadi**, shuning uchun shunday obyekt ichidagi funksiyalarni chaqirish mumkin. Ammo obyektlar haqida batafsil tushuntirish ushbu kitobning doirasidan tashqariga chiqish kerak, bu mavzuga qiziquvchilar Pythonda obyektlarni boshqarishni alohida o'rganishlarini tavsiya qilamiz. Hozircha shuni bilib olish kifoya: ro'yxatga yangi element qo'shish uchun **ro'yxat_nomi.append(qo'shilayotgan qiymat)** sintaksisini ishlatish kerak.

12 **34** Endi 18~20-qatorlarni ko'rib chiqamiz. Bu yerda **while** yordamida yozilgan takroriy sikl mavjud. Ammo sikl qachongacha davom etishini belgilovchi shart keltirilgan: **len(numbers) != 0**. Biz allaqachon "==" operatorining ma'nosini bilamiz. Biroq, "!=" operatorining ma'nosi nima? Bu aynan "==" operatorining aksi bo'lib, ikki qiymat bir-biriga teng emasligini anglatadi. Shunday qilib, ushbu kodning ma'nosini tushuntiradigan bo'lsak, **len(numbers)** funksiyasining qaytariluvchi(return) qiymati 0 ga teng bo'lmaguncha sikl davom etadi.

Endi **len** funksiyasi nima ekanligini tushunishimiz kerak. Uning ishlatilishini ko'rib chiqamiz, bunda qiziqarli jihat bor. Funksiyaning oldida uni qaysidir joydan (masalan, biror kutubxona nomidan, ro'yxat nomidan yoki o'zgaruvchi nomidan) chaqirilganini ko'rmaymiz. Bu nimani anglatadi? Ya'ni, **bu funksiya biror joyga tegishli emas, balki Python tilining ichida standart tarzda o'rnatilgan ichki funksiya ekanligini bildiradi**. Shu sababli, bu funksiyani hech qanday maxsus ko'rsatmalarsiz, faqat nomini yozib chaqirsa bo'ladi, va biz bunday funksiyalarni maxsus "**ichki funksiyalar**" deb ataymiz. Shunday qilib, **len** funksiyasi ichki funksiya bo'lib, uning vazifasini nomidan ham tushunish mumkin: **length** (uzunlik) so'zining qisqartmasi sifatida olingan. U ma'lum bir ma'lumotning hajmini (ya'ni, uning uzunligini) qaytaradi. Shu sababdan, bu yerda **len** funksiyasiga uzatilgan ma'lumot qiymati **numbers** deb nomlangan ro'yxatdir. Natijada, bu ro'yxatdagi elementlar jamini ko'rsatadi. Masalan, agar 25-qator kodi bo'yicha foydalanuvchi ikkita sonli qiymat kiritgan bo'lsa, len funksiyasi yordamida ro'yxatda hozir 2 ta element borligini aniqlash mumkin. Agar foydalanuvchi uchta sonli qiymat kiritgan bo'lsa, len funksiyasi hozirgi elementlar jamini 3 deb ko'rsatadi.

Yuqorida alohida-alohida tushunilgan ma'lumotlarni birlashtirib, ushbu kodni aniqroq talqin qiladigan bo'lsak, ***len(numbers) != 0*** sharti bilan ishlaydigan **while** sikli, oxir-oqibat, **numbers** ro'yxatida birorta ham element qolmaguncha takrorlashni bildiradi. Foydalanuvchi o'zi xohlagancha qiymatlarni kiritgani sari **numbers** ro'yxati elementlar bilan to'ldiriladi. Ammo, "qanday qilib ro'yxatda birorta ham element qolmasligi mumkin?" degan savol paydo bo'lishi mumkin. Bu savolning javobini esa 19~20-qatorlarni tahlil qilib topish mumkin.

12 19~20-qatorlarda foydalanuvchi tomonidan **numbers** ro'yxatiga kiritilgan sonli qiymatlarini birma-bir olib, ularning yig'indisini hisoblash uchun yozilgan kod mavjud. Yuqorida biz ro'yxat bilan ishlashning uch xil usulini o'rgandik (yangi qiymat qo'shish, mavjud qiymatni o'qish, mavjud qiymatni o'chirish), va ulardan bittasi — yangi qiymat qo'shishni ko'rdik. Endi esa ro'yxatda saqlangan qiymatlarni o'qish usulini o'rganish vaqti keldi. Buni aniqroq tushunish uchun avvalo, sintaksis va ro'yxatning indeks(index) orqali murojaat qilish degan tushunchasini o'rganish kerak. Ro'yxatning ma'lum bir elementini o'qish uchun quyidagi sintaksis qo'llaniladi:

ro'yxat_nomi[indeks]

Quyidagi misolda ko'rganimizdek, ro'yxatga element qo'shilgan vaqtdan boshlab (ro'yxatni aniqlash va boshlang'ich qiymat berishda yoki yangi element qo'shishda), Python ichki tizimi avtomatik tarzda ro'yxat ichidagi har bir elementning tartib raqamini belgilaydi. **Bu tartib 0 dan boshlanadi va keyingi elementlar uchun birma-bir ortib boradi.** Bu indeks qiymatlari ichki tizimda mavjud bo'lib, bizga ko'rinmaydi, lekin ular yordamida ro'yxat ichidagi har bir elementning qiymatini o'qib olishimiz mumkin.

words = ["apple", "banana", "grape"]
Indeks
qiymatlari :
0
1
2

Agar biz **words** ro'yxatidagi ikkinchi o'rinda turgan "banana" elementini (qiymatini) o'qimoqchi bo'lsak, ro'yxatdagi ma'lum bir o'rindagi elementni o'qish sintaksisiga muvofiq, **words[1]** deb yozib, uni o'qib olishimiz mumkin. Quyidagi rasmda ko'rsatilganidek, ro'yxat ichidagi qiymatlar va ularning indeks qiymatlari real vaqt rejimida vizual ko'rinishda tasvirlangan. Bizning misolimizda bajarilish natijasi ekranda ko'rinadi: chap yuqori burchakda **numbers** ro'yxatida 2 va 3 qiymatlari o'sha tartibda joylashganligini ko'rishingiz mumkin. Ya'ni, foydalanuvchi avval 2 sonini kiritgan, keyin esa 3 sonini kiritgan.

Qo'shmoqchi bo'lgan raqamni kiriting. Agar siz hozirgacha kiritgan qiymatlar yig'indisini bilmoqchi bo'lsangiz, '=' kiriting. Agar siz ilovadan chiqmoqchi bo'lsangiz, 'x' kiriting.



numbers

0	2
1	3

result

0

12 19-qatorda **numbers[0]** sintaksisi orqali **numbers** ro'yxatidagi birinchi elementni o'qib olib, uning qiymatini **result** deb nomlangan o'zgaruvchiga saqlash vazifasi bajariladi. Keyingi, ya'ni 20-qator kodida nima sodir bo'ladi? Bu yerda **numbers** ro'yxati(obyekti) ichida joylashgan **numbers.pop()** funksiyasi chaqirilmoqda. Funksiyaning nomidan ham tushunish mumkinki, **pop** funksiyasi ro'yxatdan ma'lum bir o'rinda joylashgan qiymatni olib, uni o'chirish uchun ishlatiladi. Ushbu funksiyani quyidagi shaklda chaqirib ishlatish mumkin: **ro'yxat_nomi.pop(o'chirilishi kerak bo'lgan elementning indeksi)**. Shunday qilib, 20-qatordagi **numbers.pop(0)** ning **ma'nosi** — ro'yxatdagi birinchi elementni (indeks 0 ga ko'ra) olib tashlashdir.

Bizning misolimizda yuqoridagi bajarilish natijasida (foydalanuvchi 2 va 3 sonlarini ketma-ket kiritgan holatda) ushbu kod bajarilgandan keyin ro'yxatdagi birinchi qiymat, ya'ni 2 ro'yxatdan o'chiriladi. Shu bilan birga, undan keyin turgan 3 soni avtomatik ravishda o'chirilgan 2 ning joyiga ko'chadi va endi birinchi o'rinda turadi. **pop(0)** funksiyasi har doim joriy vaqtda ro'yxatning oldingi (birinchi) o'rnida joylashgan qiymatni olib tashlashni anglatadi. Yuqoridagi kodni Entry muhitida ishga tushirib ko'ring. Uni matnda o'qib tushunishdan ko'ra, vizual ko'rib tushunish osonroq. Chunki Entry ushbu jarayonlarni bosqichma-bosqich ko'rsatib, real vaqt rejimida vizual ko'rsatmalar orqali tushunishni osonlashtiradi.

12 18~20-qator kodlarini umumlashtirib qayta tahlil qiladigan bo'lsak, **numbers** ro'yxatida saqlanayotgan elementlarni birma-bir o'qib, avvalgi yig'indi qiymatiga (ya'ni, **result** o'zgaruvchisiga) qo'shish, so'ng o'qilgan qiymatni ro'yxatdan o'chirib tashlash jarayonlarini bajaradi. Ushbu jarayon, ro'yxatda hech qanday element qolmaguncha davom ettiriladi. Bu kodning maqsadi — foydalanuvchi tomonidan kiritilgan barcha qiymatlarning yakuniy yig'indisini hisoblashdir. Shu bilan birga, yig'indi hisoblanayotganda ro'yxatni tozalab, bo'sh holatga keltirish ham amalga oshiriladi.

Boshidan qaytib, bizning maqsadimiz foydalanuvchidan cheksiz ravishda sonlarni qabul qilib, ushbu kiritilgan barcha sonlarning yig'indisini hisoblash funksiyasiga ega bo'lgan dasturni muvaffaqiyatli yaratish edi. Bu maqsadga erishildi. Albatta, ushbu funksiyani faqat yuqoridagi usulda dasturlash kerak degan qoidalar yo'q. Bir xil funksiyani bir necha usul bilan amalga oshirish mumkin, va hozirgi kod shulardan faqat bittasidir. Aslida, bu kod Entry-Python emas, balki original Python bo'lganida, yanada oddiy va samaraliroq kod yozish mumkin edi. Biroq, biz Entry-Pythonning imkon bergan sintaksis chegaralari doirasida kodlashimiz kerak, va shu bilan birga sizga ro'yxat bilan ishlashning turli usullarini ko'rsatish maqsadida aynan shunday tarzda kod yozilganini ham inobatga oling.

3.7 Tasodifiy son (Random)

Tasodifiy son (yoki random) funksiyasidan foydalanish faqat bitta funksiya chaqiruviga asoslangani sababli, hozirgacha berilgan ma'lumotlarni yaxshi tushungan va kuzatib kelgan odamlar uchun umuman qiyinchilik tug'dirmaydi. Shunday qilib, ushbu bo'limda oddiy va qiziqarli bir o'yin yaratamiz, bunda tasodifiy son funksiyasidan foydalanamiz.

O'yin nomi "Men o'ylagan sonni top!" bo'lib, kompyuter 1 dan 50 gacha bo'lgan tasodifiy bir sonni o'ylaydi, va biz ushbu sonni imkon qadar kam urinishda topishimiz kerak bo'ladi. O'yinni yanada qiziqarliroq qilish uchun biz foydalanuvchining kiritgan taxminiy soni to'g'ri javobga qanchalik yaqin ekanligi haqida doimiy ravishda maslahatlar beradigan mexanizmni qo'shamiz.

Entry-Python

```
1. # (1)Entrybot's Python code
2. import Entry
3.
4. com_num = 0
5. my_num = 0
6. try_total = 0
7.
8. def when_start():
9.     com_num = random.randint(1, 50)
10.
11.     while True:
12.         Entry.input("Men 1 dan 50 gacha bo'lgan sonni o'ylab qo'ydim, uni topishga
harakat qiling! Siz ularni necha martada to'g'ri topasiz?")
13.         my_num = Entry.answer()
14.         try_total += 1
15.
16.         if my_num == com_num:
17.             Entry.print("Bingo! " + try_total + "-urinishda topib oldingiz-ku!")
18.             Entry.stop_code("all")
19.         else:
20.             if my_num < com_num:
21.                 Entry.print(my_num + " dan katta son")
22.             else:
23.                 Entry.print(my_num + " dan kichik son")
24.             Entry.wait_for_sec(2)
```

Blokli kodlash



Ijro natijasi



 9-qatorida tasodifiy son yaratish uchun **randint** funksiyasidan foydalanilgan. Ushbu funksiya **random** modulida (yoki kutubxonasida) joylashgan bo'lib, asl Python'da ushbu modulni chaqirish uchun kodning eng yuqori qismida **import random** qatori qo'shilishi kerak bo'lardi. Biroq, Entry-Python'da ushbu jarayonsiz ham ushbu funktsiyani to'g'ridan-to'g'ri chaqirib foydalanish mumkin. *Funksiyaga uzatilgan qiymatlar qaysi oraliqda tasodifiy son kerakligini aniq belgilash uchun berilgan: 1 dan 50 gacha bo'lgan butun sonlar oraliqni anglatadi, shuning uchun **randint(1, 50)** chaqiruviga boshi va oxiri sifatida ikkita qiymat funksiya argumentlariga uzatilgan.*

 14-qatorida **try_total += 1** qatorida notanish **+=** operatori ishlatilgan, lekin aslida bu operatorning ishlatilishi oldingi bo'limda ro'yxatlar bilan ishlash misolida ko'rsatilgan kodga o'xshaydi. U yerda umumiy summani hisoblashda avvalgi qiymatga yangi qiymat qo'shib borilgan edi. Shunday qilib, agar kodni **try_total = try_total + 1** shaklida yozsangiz, u xuddi shunday ishlaydi, va bu kodni qisqartirilgan holda **try_total += 1** shaklida yozish ham aynan shu ma'noni beradi. Ya'ni, **try_total** ning oldingi qiymatiga ketma-ket 1 qo'shib borilsin degan ma'noni anglatadi.

Boshqa kodlar ilgari o'rganilgan bo'lib, ularni tushunishda katta qiyinchilik bo'lmasligi kutiladi. Shuning uchun qo'shimcha izohlar berishga hojat yo'q, va endi ushbu o'yinni yanada qiziqarli qilish uchun uni rivojlantirish sizning vazifangiz bo'lib qoladi.

3.8 Funksiya (Function)

Nihoyat, endi bu bo'limning oxirgi qismi — funksiya mavzusi qoldi. Funktsiyalarni biz hozirgacha faqat kimdir biz uchun oldindan yaratib qo'ygan funksiylarni chaqirib ishlatib ko'rganmiz. Ammo o'zimizning funksiylari(foydalanuvchi funksiylari)mizni hali hech qachon yaratib ko'rmaganmiz. Endi esa shu funksiylarni o'zimiz yozishni sinab ko'ramiz. 3.7-bo'lim da yozgan o'yin kodimizning ayrim qismlarini funksiya sifatida alohida amalga oshirib, xuddi shu funksionallikni qayta bajarish orqali funksiylarni o'rganamiz.

Avval kodning qaysi qismini foydalanuvchi funksiya(o'zimizning shaxsiy funksiya) sifatida alohida ajratib yaratishni aniqlashimiz kerak. Masalan, foydalanuvchi kiritgan son bilan kompyuter o'ylagan sonidan farqini solishtirish qismiga e'tibor qaratamiz. Shu qismni ajratib, o'zimizning foydalanuvchi funksiya(mizni) yaratamiz va ikki sonni solishtirish vazifasini shu funksiya topshiramiz.

Entry-Python(foydalanuvchi funksiya)

```
1. # (1)Entrybot's Python code
2. import Entry
3.
4. com_num = 0
5. my_num = 0
6. try_total = 0
7.
8. def when_start():
9.     com_num = random.randint(1, 50)
10.
11.     while True:
12.         Entry.input("Men 1 dan 50 gacha bo'lgan sonni o'ylab qo'ydim,
13.             uni topishga harakat qiling! Siz ularni necha martada to'g'ri topasiz?")
14.         my_num = Entry.answer()
15.         try_total += 1
16.
17.         if my_num == com_num:
18.             Entry.print("Bingo! " + try_total + "-urinishda topib oldingiz-ku!")
19.             Entry.stop_code("all")
20.         else:
21.             if my_num < com_num:
22.                 Entry.print(my_num + " dan katta son")
23.             else:
24.                 Entry.print(my_num + " dan kichik son")
25.             Entry.wait_for_sec(2)
```

Blokli kodlash(foydalanuvchi funksiyasiz)



Entry-Python(foydalanuvchi funksiyasi bilan)

```
1. # (1)Entrybot's Python code
2. import Entry
3.
4. com_num = 0
5. my_num = 0
6. try_total = 0
7.
8. def compare_num(num1, num2):
9.     if my_num == com_num:
10.         Entry.print("Bingo! " + try_total + "-urinishda topib oldingiz-ku!")
11.         Entry.stop_code("all")
12.     else:
13.         if my_num < com_num:
14.             Entry.print(my_num + " dan katta son")
15.         else:
16.             Entry.print(my_num + " dan kichik son")
17.         Entry.wait_for_sec(2)
18.
19. def when_start():
20.     com_num = random.randint(1, 50)
21.
22.     while True:
23.         Entry.input("Men 1 dan 50 gacha bo'lgan sonni o'ylab qo'ydim, uni topishga
harakat qiling! Siz ularni necha martada to'g'ri topasiz?")
24.         my_num = Entry.answer()
25.         try_total += 1
26.
27.         compare_num(my_num, com_num)
```

Blokli kodlash(foydalanuvchi funksiyasi bilan)



Ijro natijasi



Biz ushbu bo'limning **birinchi bob** da funksiyalarni qanday yaratishni o'rganganimiz sababli, agar hali funksiyalarni yaratish sintaksisini tushunishda qiyinchiliklar bo'lsa, o'sha bo'limga qaytib, uni takrorlashni tavsiya qilamiz.

12 8~17-qatorlardagi kod **avvalgi dastur kodi** ning 16~24-qatorlarini ajratib, **compare_num** deb nomlangan foydalanuvchi funksiyasini yaratishga bag'ishlangan. Ushbu funksiya ikkita sonli qiymatlarini parametr sifatida qabul qiladi va funksiya ichida bu ikkita sonlarning kattaliklarini solishtirish natijasiga ko'ra shartli bajarish natijasini ko'rsatadi. Agar funksiyalarni yaratish va ulardan foydalanish usulini allaqachon o'zlashtirgan bo'lsangiz, bu qismni tushunishda hech qanday qiyinchilik bo'lmaydi.

Biroq, ushbu foydalanuvchi funksiyasi amalda unchalik samarali emas, chunki **funksiyaning asosiy maqsadi — dastur kodidagi turli qismlarda takrorlanadigan funkcionallikni bitta funksiya orqali almashtirib, umumiy kodni soddalashtirish, shu bilan birga kodning o'qilishiga qulaylik yaratish va keyinchalik yana qaytadan yuzaga kelishi mumkin bo'lgan kodni tahrirlash jarayonlarini osonlashtirishdir.** Ammo hozir yaratilgan foydalanuvchi funksiyasi bu maqsadga unchalik mos emas va asosan o'rganish maqsadida — o'z funksiyamizni yaratib, undan foydalanishni sinab ko'rish uchun sun'iy misol sifatida yaratilganini aytish mumkin.

Bundan tashqari, **foydalanuvchi funksiyasini ko'proq joyda ishlatish uchun imkon qadar umumiy funkcionallikni amalga oshirgan ma'qul.** Masalan, hozirgidek kattalikni solishtirish + solishtirish natijasini ekranga chiqarishni birlashtirishdan ko'ra (bunday holatda funksiya faqat muayyan kodlarda cheklangan holda foydalanish mumkin bo'ladi), faqatgina kattalikni solishtirish funkcionalligini alohida bir funksiya sifatida amalga oshirish yaxshiroq bo'lardi. Ammo, afsuski, hozirgi Entry-Python darajasida bunday funkسيyani yaratishning ba'zi cheklovlari bor (masalan, funksiyaning natija qiymatini funksiya chaqiruvchisiga qaytarish (return) funksiyasi amalga oshirilmagan). Shuning uchun bunday ko'rinishda amalga oshirilganligini hisobga olish kerak.

Bundan tashqari, yuqorida ko'rsatilgandek funksiya yozilganidan so'ng “Boshlash” tugmasini bosib dasturni ishga tushirib, keyin uni yopganingizda, yaqinda yozgan funksiyangizning kutilmaganda yo'qolgandek ko'rinishi mumkin. Aslida, ushbu funksiya avtomatik ravishda blokli kodlashning “Funksiyalar” bo'limiga ro'yxatdan o'tadi va kod u yerga ko'chiriladi. Bu holat, afsuski, Entry-Pythonda funksiya yozishda foydalanuvchi uchun chalkashliklar keltirib chiqarishi mumkin bo'lgan jihatlardan biridir.

4. Entry-Python yordamida dasturlashning asosiy tushunchalarini o'rganamiz

Ushbu bobdan boshlab, tilning o'ziga xos sintaksisidan tashqari, dasturlash paradigmalari deb ataladigan tushunchalar va ularning ichida asosiy elementlarni Entry-Python yordamida o'zgina bo'lsa ham o'rganib, Entry-Python bo'yicha o'quv jarayonini yakunlashni maqsad qilamiz.

Paradigma umumiy atama bo'lib, lug'at ma'nosida fikrlashni tashkil etuvchi idrok tizimi sifatida ta'riflanishi mumkin. Dasturlash dunyosida bu xuddi shunday ma'noga ega. Dasturlashda siz ma'lum bir yondashuv yoki fikr doirasiga amal qilasiz va shu doirada kod yozasiz. Ushbu tushunchani to'liq anglash qiyin bo'lishi mumkin, shuning uchun uni oldingi boblarda keltirilgan xorijiy tilga o'xshatib tushuntirishga davom etsak, avval tilning sintaksis qoidalarini o'rgangan bo'lsak, endi qanday janr yoki uslubda yozish haqida o'ylash kerak bo'ladi. Masalan, yozuvchilikda insho va film/yangi ssenariy yozish janrlari bir-biridan sezilarli darajada farq qiladi. Xuddi shunday, dasturlashda ham oxirgi maqsadga mos keladigan dastur uchun ma'lum bir paradigma tanlanadi va shu forma hamda doira ichida kod yoziladi. Ammo yozuvchilikda *“janrni buzish”* yoki *bir nechta janrni birlashtirish mumkin bo'lgani kabi, dasturlashda ham bir nechta paradigmani aralashtirib dasturlash mumkinligini bilish foydali bo'ladi.*

Aslida, bu paradigmalarning tushunchalari Entry blokli kodlash ichida allaqachon mavjud, lekin biz bu vaqtgacha buni anglamasdan kod yozib kelganmiz. Agar siz: “Men faqat blokli kodlash bilan shug'ullanib, dasturlashni shu yerdan yakunlayman”, deb qaror qilsangiz, bu tushunchalarni bilmasdan ham davom etishingiz mumkin. Bunda ham blokli kodlashdan zavqlanishingizga hech narsa to'sqinlik qilmaydi. Ammo biz endi blokli kodlash dunyosidan o'tib, matnli kodlash (text coding) dunyosiga qadam qo'ymoqdamiz. Ushbu dunyoda kod yozish uchun qo'shimcha bilimlar talab qilinadi. Ushbu bilimlarni bilgan holda kodlash bilan bilmasdan kodlashning farqi katta bo'ladi. Matnli dasturlash dunyosida samaraliroq ishlash uchun ushbu bilimlar asos yaratib beradi. Bu dunyoda boshqa hamkasblar bilan hamkorlik qilish odatiy hol bo'lib, ular bilan muvaffaqiyatli ishlash uchun asosiy bilimlarni o'rganishni ushbu jarayonning bir qismi sifatida qabul qilish mumkin.

4.1 Ketma-ket/Parallel bajarish (Serial/Parallel)

Ketma-ket/parallel bajarish tushunchasi aslida dasturlash paradigmasi emas. Oddiy qilib aytganda, bu bir nechta vazifalarni ketma-ket yoki bir vaqtning o'zida ularni bajarish tartibida farqdir.

Ketma-ket bajarish (Serial processing) — kod qatorlarini birin-ketin bajarishni anglatadi. Bu tushuncha kompyutersiz(unplugged) kodlash ta'limida ham eng asosiy mavzu bo'lib, aslida hammamiz yaxshi biladigan oddiy tushuncha, shuning uchun alohida tushuntirishga hojat yo'q. Parallel bajarish (Parallel processing) esa kompyuter muhandisligi nazariyasida murakkabroq tushuncha bo'lib, yangi boshlovchilar uchun tushunish qiyin bo'lishi mumkin. Bundan tashqari, Entry-Pythonda bunday darajadagi parallel bajarish qo'llab-quvvatlanmaydi, shuning uchun faqat asosiy tushunchani bilib o'tish yetarli. **Parallel bajarish bir nechta vazifani bir vaqtning o'zida bajarish** usuli sifatida tushuntiriladi. Bunda haqiqiy vazifalar butunlay mustaqil holda bajariladimi (Parallelism, paralellik), yoki vazifalar go'yoki bir vaqtda bajarilayotgandek ko'rinish hosil qiluvchi texnika (Concurrency, sinxronlik) yordamida bajariladimi, bu ikki tushunchani farqlash muhim. Entry-Python ikkinchi usulga — sinxronlikka tayanadi.

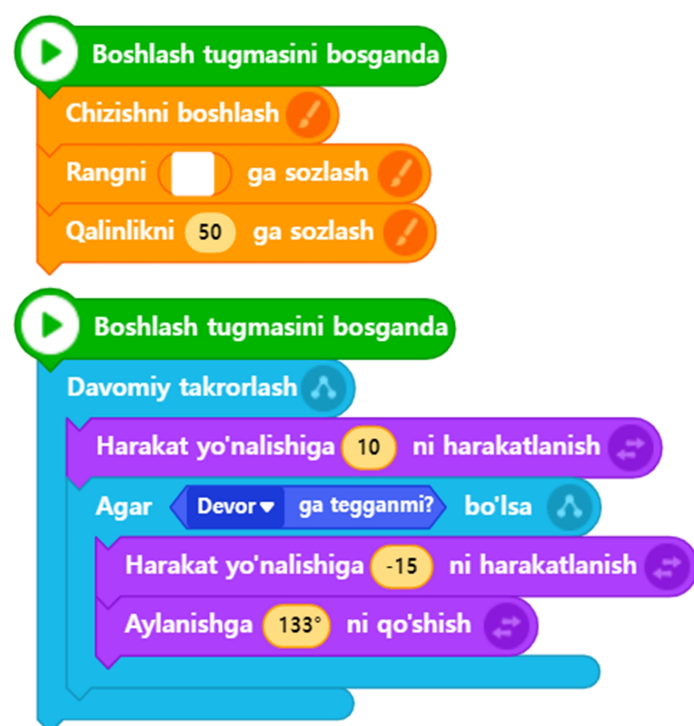
Entry-Pythonda sinxronlik(concurrency) bilan bog'liq dasturlash texnikalari (Thread(Oqimlar), Multi-processing(Ko'p-jarayonlik) va boshqalar) qo'llab-quvvatlanmagani sababli, bu usullardan foydalanib kod yozishni yoki sinxronlikka oid dasturlash usullarini o'rganishni imkonsiz qiladi. Kitobning boshida ham ta'kidlanganidek, bu Entry-Pythonning cheklovidir. Shu bilan birga, Entry-Python dastlab blokli kodlashdan birinchi matnli dasturlashga o'tganlar uchun mo'ljallangan bo'lib, murakkab darajadagi o'rganish maqsad qilinmagan. Shuning uchun, asosiy tushunchalarni o'zlashtirib, texnik tafsilotlarga chuqur kirmasdan o'tishning o'zi kifoya. Haqiqatan ham, bu darajadagi tushunchalar ham o'z ahamiyatiga ega ekanligini bilish foydali bo'ladi.

Entry blokli kodlashida, quyidagi misolda ko'rganingizdek, biz allaqachon ko'p bor parallel bajarishni kodlab ko'rganmiz.

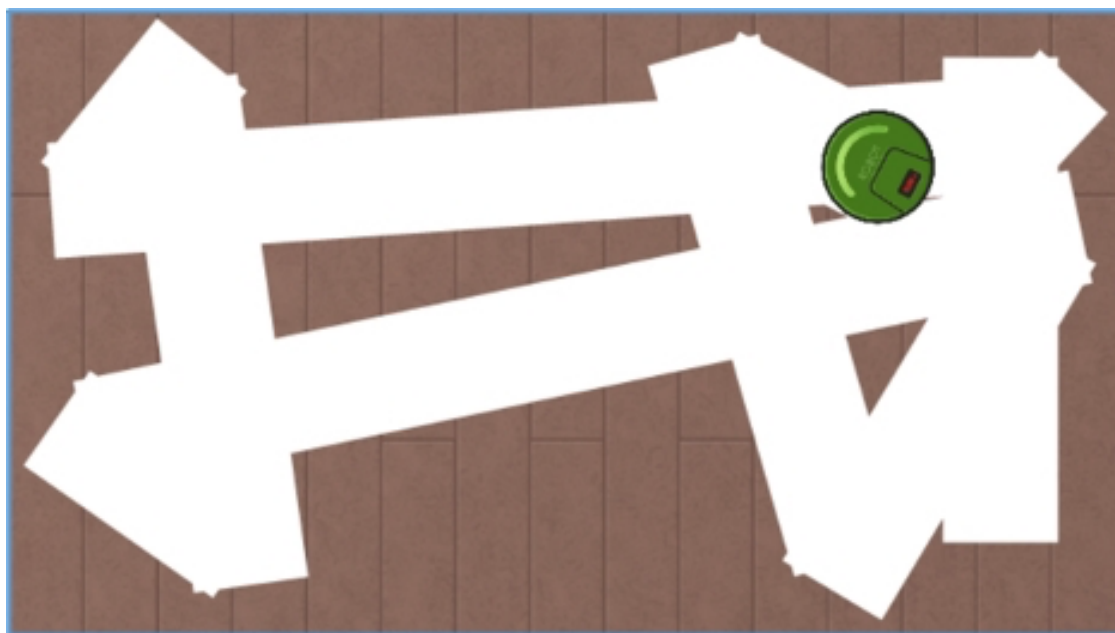
Entry-Python

```
1. # Changyutgich robot(1)'s Python code
2.
3. import Entry
4.
5. def when_start():
6.     Entry.start_drawing()
7.     Entry.set_brush_color_to("#FFFFFF")
8.     Entry.set_brush_size(50)
9.
10. def when_start():
11.     while True:
12.         Entry.move_to_direction(10)
13.         if Entry.is_touched("edge"):
14.             Entry.move_to_direction(-15)
15.             Entry.add_rotation(133)
```

Blokli kodlash



Ijro natijasi



“Boshlash tugmasini bosganda” deb nomlangan blokni bir obyektida bir necha marta ishlatib, mustaqil ko‘rinadigan vazifalarni go‘yoki bir vaqtning o‘zida bajarilayotgandek kodlagan tajribamiz bor edi. Bu orqali biz turli dasturlashning asosiy tushunchalaridan foydalanib kelganmiz, buni o‘zimiz sezmagani holda bajargan bo‘lsak ham.

Shu sababli, Entry-Pythonda parallel bajarish blokli kodlashda qilgan ishlarimizdan unchalik farq qilmaydi. Faqat yuqoridagi misolda ko‘rsatilganidek, **when_start** funksiyasini bir necha marta ishlatish mumkin. Foydalanuvchi dastur ishga tushishi uchun “Boshlash” tugmasini bosganda, har bir **when_start** funksiyasi bir vaqtning o‘zida chaqiriladi. Shu tarzda, har bir funksiyada bajarilishi kerak bo‘lgan vazifalarni mustaqil ravishda tasvirlab, ularni go‘yoki bir vaqtda bajarilayotgandek ko‘rinish hosil qilish mumkin. Bu usulni kodlashda va dasturda samarali foydalanish kifoya qiladi.

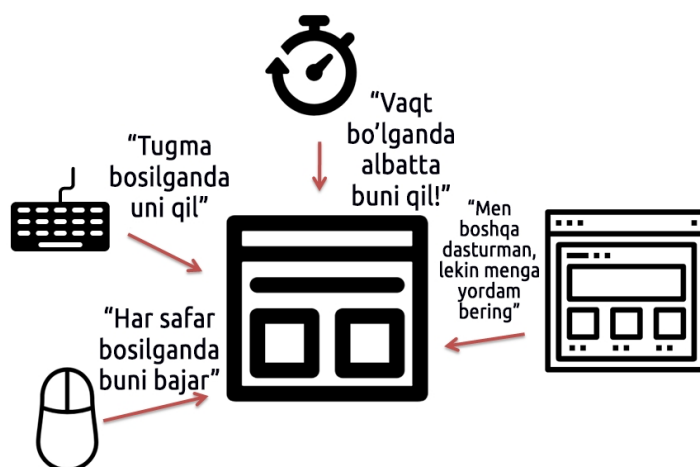
4.2 Jarayonga yo'naltirilgan (Procedural)

Dasturlash paradigmalari orasida “jarayonga yo'naltirilgan” (procedural) paradigma mavjud. Bu dasturlashning eng an'anaviy paradigmasi bo'lib, dastlab paydo bo'lgan paradigmalor orasida eng birinchisi hisoblanadi. Biz bilib-bilmay Entry blokli kodlashda ham ushbu paradigmani qo'llab kelganmiz. [3.8-bo'lim](#) da funksiyalarning zaruriyati va maqsadini yaxshi tushungan edik. Dastur ichida qayta foydalanish imkoniyati (reusability) yuqori bo'lgan kodlarni funksiyalarga ajratib, ularni chaqirish orqali kod yozish usuli jarayon yo'naltirilgan paradigma sifatida tanilgan. Ilgari funksiya(function) atamasi o'rniga “jarayon(procedure)” yoki “sub-programma(sub-routine)” deb atalgan. Shu sababli, jarayon yo'naltirilgan dasturlashning inglizcha nomi “procedural” bo'lib kelgan.

Jarayonga yo'naltirilgan paradigmaning misollari oldingi bo'limlarda funksiyalarni o'rganishda ko'rgan misollardan unchalik farq qilmaydi, shuning uchun o'sha misollarni bu yerda ham qo'llash kifoya.

4.3 Hodisaga asoslangan (Event-Driven)

Keling, **hodisaga asoslangan**(Event-Based) yoki **hodisaga yo'naltirilgan dasturlash paradigmasi** haqida gaplashamiz. Ushbu paradigma keng qo'llanila boshlagan davr operatsion tizimlar(OS: Operating System) interfeyslari **konsol**(Console) matnli buyruqlari orqali ishlashdan **grafik foydalanuvchi interfeysi**(GUI: Graphic User Interface)ga o'tish davriga to'g'ri keladi. Boshqacha aytganda, GUI muhitiga mos dasturlar kerak bo'ldi, va dastur ishlab chiqish usullari ham shu ehtiyojlarga mos ravishda rivojlandi. **GUI muhiti** deganda grafik asosida ishlaydigan tizim tushuniladi. Masalan, kompyuter yoki smartfonlarda biz bosish yoki teginish orqali tanlovlar va boshqaruvlar amalga oshiramiz. Bugungi kunda bu kompyuterni boshqarishning keng tarqalgan usuli bo'lib, ushbu paradigmadan foydalanishni tushunish qiyinchilik tug'dirmaydi. Shunga qaramay, hodisaga yo'naltirilgan dasturlash konsepsiyasini quyidagi rasm yordamida yaxshiroq tushunishga harakat qilaylik.



Manba: [Kamang's IT Blog](#)

Hozir ekranda ko'rinib turgan dasturimda biror harakatni qo'zg'atuvchi(trigger) barcha holatlar hodisa(event) deb ataladi. Masalan, foydalanuvchi ekranda ma'lum bir rasmlı tugmani yoki belgini bosgan payt, klaviaturada ma'lum bir tugmachasi(harf, maxsus belgi, yo'nalish tugmalari va boshqalar)ni bosgan payt, sichqonchaning ma'lum bir tugmachasi(chap, o'rta, o'ng)ni bosgan payt, belgilangan vaqtga yetgan payt yoki boshqa bir dastur mening dasturimni chaqirgan payt va shunga o'xshash holatlar.

Ya'ni, dasturchi bunday hodisalar sodir bo'lishini oldindan taxmin qilib, har bir hodisa sodir bo'lganda qaysi vazifani bajarish kerakligini ko'rsatadigan tarzda kod yozishni **hodisaga asoslangan dasturlash paradigmasi** deb atash mumkin. Biz allaqachon Entry platformasida shu usulda kod yozish tajribasiga egamiz, shuning uchun ushbu tushunchani anglash yoki amalda qo'llashda qiyinchilik bo'lmaydi, deb o'ylayman. Faqatgina, Entry-Pythonda bu kabi hodisalar uchun qanday funksiyalardan foydalanish kerakligini bilishimiz kifoya.

Quyidagi jadvalda Entryda mavjud bo'lgan hodisalar turlari va ushbu hodisalarga mos keladigan, oldindan aniqlangan(kelishilgan) **kolbek(callback) funksiyalar** (3.1-bo'limdagi kolbek funksiyalarga qarang) ro'yxatini ko'rishingiz mumkin.

Hodisa turi	Blokli kodlashdagi bloklar	Matnli kodlash uchun kolbek funksiyasi
Ekrandagi boshlash tugmasini bosganda	 Boshlash tugmasini bosganda	def when_start():
Klaviaturadagi tugmacha bosilganda	 q ▼ tugmachani bosganda	def when_press_key("q"):
Sichqoncha bosilganda	 Sichqonchani bosganda	def when_click_mouse_on():
Sichqonchani bosishni qo'yib yuborganda	 Sichqonchani bosishni qo'yib yuborganda	def when_click_mouse_off():
Obyektni bosganda	 Obyektni bosganda	def when_click_object_on():
Obyektni bosishni qo'yib yuborganda	 Obyekt bosishni qo'yib yuborganda	def when_click_object_off():
Yangi sahna boshlanganda	 Sahna boshlanganda	def when_start_scene():
Obyektning nusxasi yaratilganda	 Nusxasi birinchi yaratilganda	def when_make_clone():

4.4 Obyektga yo'naltirilgan (Object-Oriented)

Nihoyat, **obyektga yo'naltirilgan(Object-Oriented, OOP) dasturlash paradigmasi** haqida gaplashaylik. Ushbu paradigma dasturiy ta'minot ishlab chiqishda eng katta ta'sir ko'rsatgan innovatsion paradigmalardan biri sifatida qaraladi, ayniqsa dastur hajmi katta yoki ko'plab dasturchilar bilan hamkorlikda ishlash zarurati bo'lganda keng qo'llaniladi. Shuning uchun uni yaxshi tushunish va bilish zarur. Biroq, bu tushunchani to'g'ri tushunish va erkin qo'llash uchun chuqur tushuncha kerak, va agar chuqurroq kirib borsangiz, ba'zi joylarda qiyinchiliklar yuzaga kelishi mumkin. Lekin bunday chuqur tushunchani keyinchalik kerakli paytda o'rganish mumkin, shuning uchun Entry-Pythonda asosiy va nazariy tushunchalarni tushunish kifoya qiladi. Bu, ayniqsa, matnli dasturlashni boshlashda katta foyda keltirishi mumkin.

Avvalgi bo'limlarda tilga olingan paradigmalarni, ularning nima ekanini bilmasdan, tabiiy ravishda o'rganganimiz kabi, Entry dasturi dastlab yaratilganidan boshlab, bizni bu paradigmalarni qiyinchiliksiz qabul qilishimiz uchun yaxshi o'ylangan dizayn mavjud bo'lgani ko'rinadi. Avvalo, biz Entryda kod yozishdan oldin, eng avvalo bajarishimiz kerak bo'lgan ish nima edi, shuni eslab ko'raylik. Ha, bu ekraniga kamida bitta obyekt qo'shish edi. Bu yerda **obyekt(Object)** atamasini joriy qilishdan boshlash, aslida, bu tushunchani tushuntirish uchun yaxshi tuzilgan ssenariy bo'lgani haqida o'ylayman.

Entryda bajarish ekranida ko'rinadigan barcha narsa obyekt hisoblanadi. **Faqat rasm yoki obyekt ekanligini aniqlash, aslida, o'sha elementga mustaqil kodlar qo'shib, uni boshqarish imkoniyati** ga bog'liq. Biz Entry blokli kodlashni boshlaganimizda, turli xil obyektlar(rasmlar, matnli obyektlar, jumladan fon obyektlari)ni olib, ularni ekraningizda joylashtirishdan boshlagandik.

Va, bu joylashtirilgan har bir obyektни jonlashtirish uchun har bir obyektga mos keladigan mustaqil kodlar yoziladi. Ba'zan, obyektlar bir-birlari bilan hamkorlik qilishlari kerak bo'lsa, obyektlar orasida bir-biriga so'rov yuboruvchi xabarlar almashish orqali o'zaro hamkorlikni yaratib, butun dasturni ishlab chiqish tarzida kod yozilgan. Hozirgacha tilga olingan barcha jarayonlar aslida obyektga yo'naltirilgan dasturlash paradigmidan foydalanish jarayoni edi va bu dasturlash tili dunyosida **obyektga yo'naltirilgan dasturlash (OOP: Object Oriented Programming)** deb ataladi va ko'pincha o'ziga xos nom kabi tilga olinadi.

Endi, misol yordamida obyektlarning o'zaro hamkorlik qilish uchun xabar almashadigan kodni ko'rib chiqaylik.

Entry-Python



```
1 # Chiroq's Python code
2
3 import Entry
4
5 def when_get_signal("yoq"):
6     Entry.change_shape("Chiroq_yoqish")
7
8 def when_get_signal("o'chir"):
9     Entry.change_shape("Chiroq_o'chirish")
```



```
1 # Svet o'chirgich's Python code
2
3 import Entry
4
5 def when_click_mouse_on():
6     Entry.change_shape_to("next")
7     Entry.send_signal("yoq")
8
9 def when_click_mouse_off():
10    Entry.change_shape_to("next")
11    Entry.send_signal("o'chir")
```

Blokli kodlash

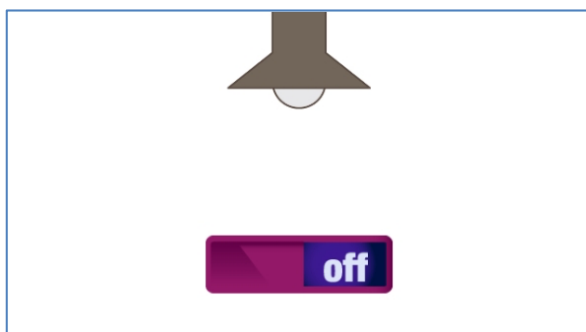


- yoq xabarini qabul qilganda
 - Chiroq_yoqish shaklga o'zgartirish
- o'chir xabarini qabul qilganda
 - Chiroq_o'chirish shaklga o'zgartirish



- Sichqonchani bosganda
 - Keyingi shaklga o'zgartirish
 - yoq xabarini jo'natish
- Sichqonchani bosishni qo'yib yuborganda
 - Keyingi shaklga o'zgartirish
 - o'chir xabarini jo'natish

Ijro natijasi



Aslida, haqiqiy matnli dasturlash asosida obyektga yo'naltirilgan dasturlashda hamkorlik jarayoni yuqorida ko'rsatilgan xabar almashish usuli o'rniga, har bir obyekt ichida mavjud bo'lgan tashqi interfeys funktsiyalarini tashqi obyektlar tomonidan to'g'ridan-to'g'ri chaqirilishi orqali amalga oshiriladi. Hozirgi tushuntirish obyektga yo'naltirilgan dasturlash paradigmasining chuqur tushunilishini talab qiladigan mavzu bo'lgani uchun, bu mavzuni bu yerda batafsil tushuntirish kitobning doirasiga kiravermaydi. Agar bu mavzu qiziqarli bo'lsa, obyektga yo'naltirilgan dasturlash paradigmalariga bag'ishlangan maxsus kitoblarni o'qish va Python tilida obyektga yo'naltirilgan dasturlashni qanday amalga oshirishni o'rganishni tavsiya qilaman.

Epilog

Shunday qilib, ushbu kitobning oxirigacha yetib kelganingiz uchun sizlarning samimiligiz va mehnatingizga katta rahmat. Endi sizlar matnli dasturlash dunyosiga kirib keldingiz, yana bir bor tabriklayman. Muallif sizlarning matnli dasturlash olamidagi qiziqarli sayohatingizni qo'llab-quvvatlash uchun [keyingi darajadagi kitob](#) ni yozishda davom etmoqda. Ushbu kitob ham dunyodagi ko'plab insonlarning fikrlash va tafakkur ufqlarini kengaytirishda, erkin bo'lishda yordam berib, ularga foyda keltirishini tilayman. 2023 yil kuzining so'nggi lahzalarida... Muallifdan

Ilova

Kodlashda yordam beruvchi qisqa tugmalar ro'yxatini taqdim etamiz. Odatda, dasturchilar tezroq ishlash uchun qisqa tugmalarni samarali qo'llashga o'rganishgan. Sizga ham qisqa tugmalardan foydalanish odatini shakllantirishni tavsiya qilamiz.

Entry-Pythonning tezkor tugmalari

Windows uchun

Funksiya	Tezkor tugma
Indentatsiya darajasini sozlash	Tab, Shift + Tab
Blok/Shakl/Ovoz/Xususiyatlar yorliqini tanlash	Alt + 1, Alt + 2, Alt + 3, Alt + 4
Oldingi/Keyingi obyektini tanlash	Alt + [, Alt +]
Blokli kodlash/Entry-Pythonga o'tish	Ctrl + [, Ctrl +]
Dasturni bajarish	Ctrl + R

Mac uchun

Funksiya	Tezkor tugma
Indentatsiya darajasini sozlash	cmd + [, cmd +] yoki Tab, Shift + Tab
Blok/Shakl/Ovoz/Xususiyatlar yorliqini tanlash	Alt + 1, Alt + 2, Alt + 3, Alt + 4
Oldingi/Keyingi obyektini tanlash	Alt + [, Alt +]
Blokli kodlash/Entry-Pythonga o'tish	Ctrl + [, Ctrl +]
Dasturni bajarish	Ctrl + R

ENTRY & PYTHON!

Python bilan sayohat qilamiz!

